CSD3009 - DATA STRUCTURES AND ANALYSIS OF ALGORITHMS

DR.S.DEVARAJU

Module 5

Graph Traversal: Breadth First Search (BFS), Depth First Search (DFS) - Minimum Spanning Tree: Prim's, Kruskal's- Single Source Shortest Path: Dijkstra's Algorithm.

https://www.javatpoint.com/data-structure-tutorial

Graph Traversal Techniques

- The previous connectivity problem, as well as many other graph problems, can be solved using graph traversal techniques
- There are two standard graph traversal techniques:
  - *Depth-First Search* (DFS)
  - *Breadth-First Search* (BFS)

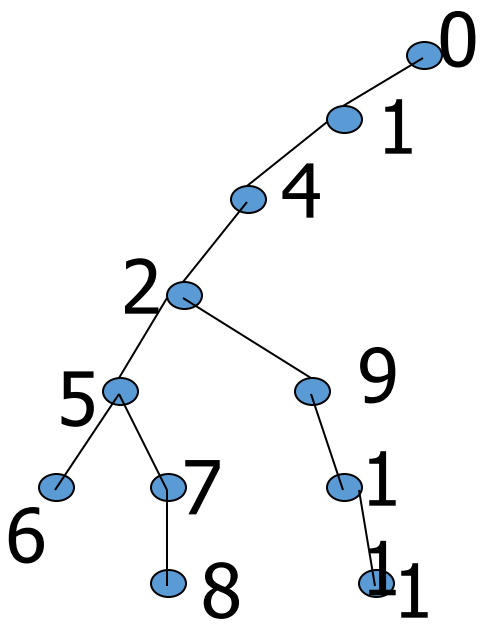
Graph Traversal (Contd.)

- In both DFS and BFS, the nodes of the undirected graph are visited in a systematic manner so that every node is visited exactly one.
- Both BFS and DFS give rise to a tree:
  - When a node x is visited, it is labeled as visited, and it is added to the tree
  - If the traversal got to node x from node y, y is viewed as the parent of x, and x a child of y

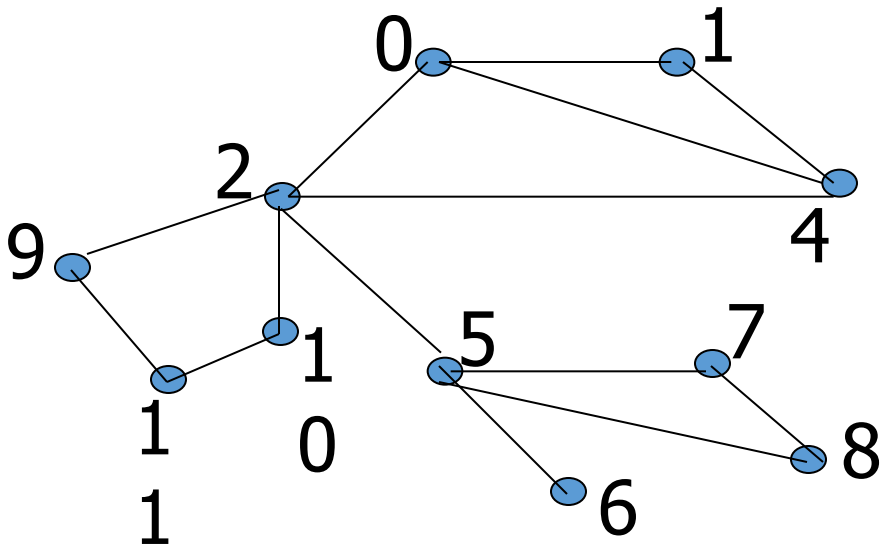
Depth-First Search

- DFS follows the following rules:
  1. Select an unvisited node x, visit it, and treat as the current node
  2. Find an unvisited neighbor of the current node, visit it, and make it the new current node;
  3. If the current node has no unvisited neighbors, backtrack to the its parent, and make that parent the new current node;
  4. Repeat steps 3 and 4 until no more nodes can be visited.
  5. If there are still unvisited nodes, repeat from step 1.

Illustration of DFS



DFS Tree



Graph G

Implementation of DFS

- Observations:
  - the last node visited is the first node from which to proceed.
  - Also, the backtracking proceeds on the basis of "last visited, first to backtrack too".
  - This suggests that a stack is the proper data structure to remember the current node and how to backtrack.

Illustrate DFS with a Stack

- We will redo the DFS on the previous graph, but this time with stacks
- In Class

DFS (Pseudo Code)

```
DFS(input: Graph G) {
  Stack S; Integer x, t;
     while (G has an unvisited node x){
     visit(x); push(x,S);
     while (S is not empty){
          t := peek(S);
          if (t has an unvisited neighbor y){
  visit(y); push(y,S); }
          else
               pop(S);
     }
  }
}
```

C++ Code for DFS

```
int * dfs(Graph G){ // returns a parent array representing the DFS tree
        int n=G.getNumberOfNodes();
    int * parent = new int[n];
    Stack S(n);  bool visited[n];
    for ( int i=0; i<n; i++)    visited[i]=false;
    int x=0;      // begin DFS from node 0
    int numOfConnectedComponents=0;
    while (x<n){     // begin a new DFS from x
      numOfConnectedComponents++;
      visited[x]=true;  S.push(x); parent[x] = -1; // x is root
      while(!S.isEmpty())     // traverse the current piece
        // insert here the yellow box from the next slide
       x= getNextUnvisited(visited,n,x);
    }
    cout<<"Graph has "<< numOfConnectedComponents<<
            " connected components\n";
    return p;
}
```

```
{
        int t=S.peek( );
        int y=getNextUnvisitedNeighbor(
                              t,G,visited,n);
        if (y<n){
              visited[y]=true;
              S.push(y);
              parent[y]=t;
        }
        else    S.pop( );
}
```

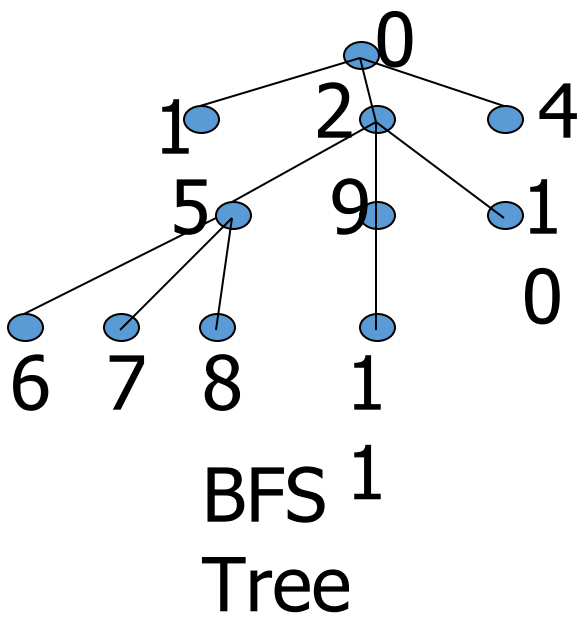
```
// Put this before dfs(…)
// returns the leftmost unvisited
// neighbor of node t. If none
// remains, returns n.
int getNextUnvisitedNeighbor(int t,
        graph G,  bool visited[],int n){
    for (int j=0;j<n;j++)
       if (G.isEdge(t,j) && !visited[j])
        return j;
    // if no unvisited neighbors left:
    return n;
}
```

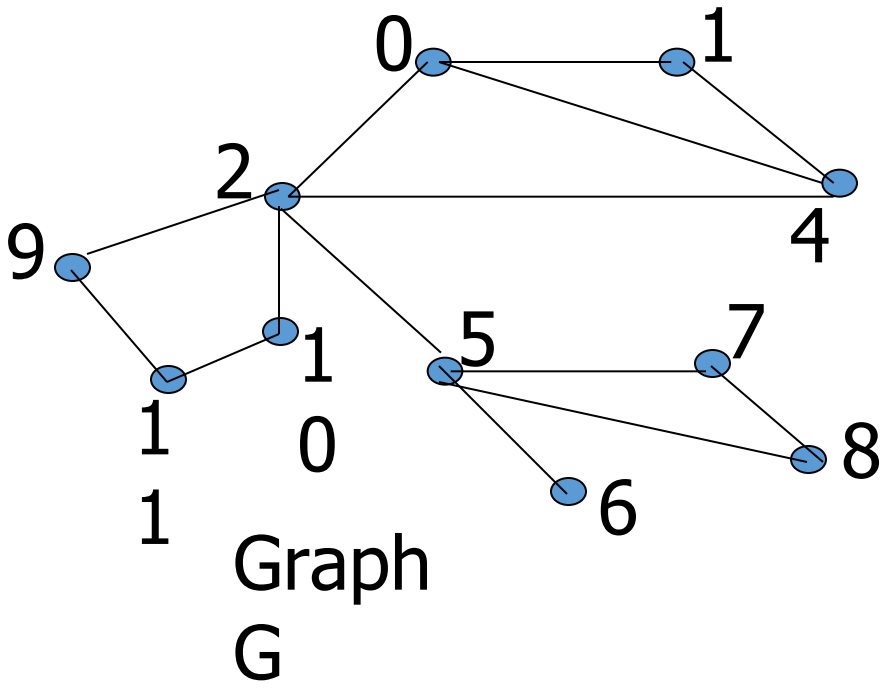
```
//Put this before dfs(…). This returns the next unvisited node, or n otherwise
int getNextUnvisited(bool visited[],int n, int lastVisited){
    int j=lastVisited+1;
    while (visited[j] && j<n)     j++;
    return j;
}
```

Breadth-First Search

- BFS follows the following rules:
  1. Select an unvisited node x, visit it, have it be the root in a BFS tree being formed. Its level is called the current level.
  2. From each node z in the current level, in the order in which the level nodes were visited, visit all the unvisited neighbors of z. The newly visited nodes from this level form a new level that becomes the next current level.
  3. Repeat step 2 until no more nodes can be visited.
  4. If there are still unvisited nodes, repeat from Step 1.

Illustration of BFS





Implementation of DFS

- Observations:
  - the first node visited in each level is the first node from which to proceed to visit new nodes.

- This suggests that a queue is the proper data structure to remember the order of the steps.

Illustrate BFS with a Queue

- We will redo the BFS on the previous graph, but this time with queues
- In Class

BFS (Pseudo Code)

```
BFS(input: graph G) {
  Queue Q;    Integer x, z, y;
  while (G has an unvisited node x) {
     visit(x); Enqueue(x,Q);
     while (Q is not empty){
          z := Dequeue(Q);
          for all (unvisited neighbor y of z){
  visit(y); Enqueue(y,Q);
          }
     }
  }
}
```

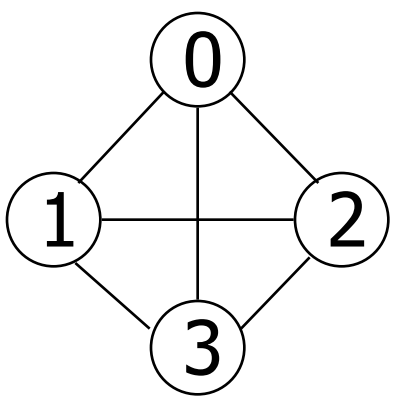
Things for you to Do

- Give a C++ implementation of BFS, using the pseudo code as your guide
- Use BFS as a way to determine of the input graph G is connected, and if not, to output the number of connected components
- Modify BFS so that the level of each node in the BFS tree is computed.
- Give another class for graph, this time using a linked lists representation.

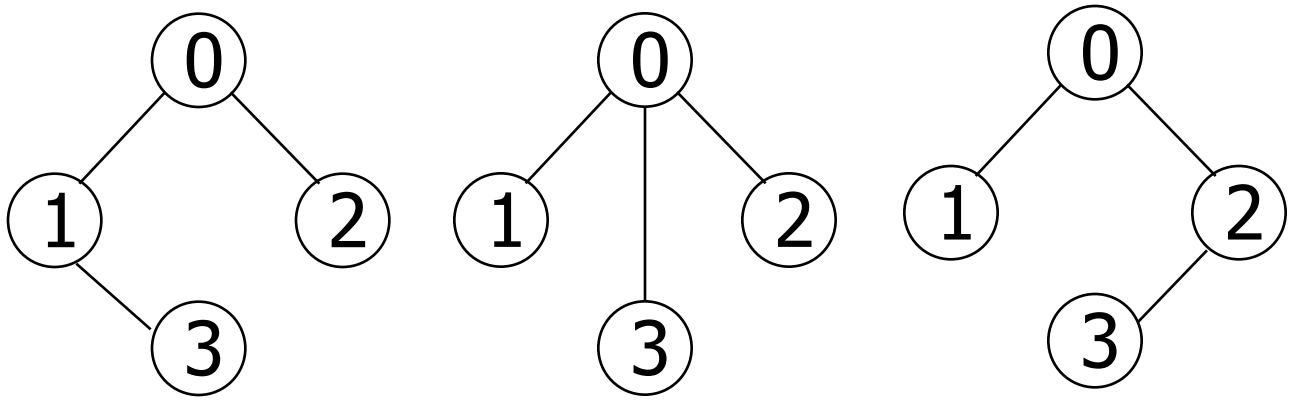
Spanning Trees

- When graph G is connected, a depth first or breadth first search starting at any vertex will visit all vertices in G
- A spanning tree is any tree that consists solely of edges in G and that includes all the vertices
- E(G): T (tree edges) + N (nontree edges) where T: set of edges used during search
  N: set of remaining edges

Examples of Spanning Tree



G_1

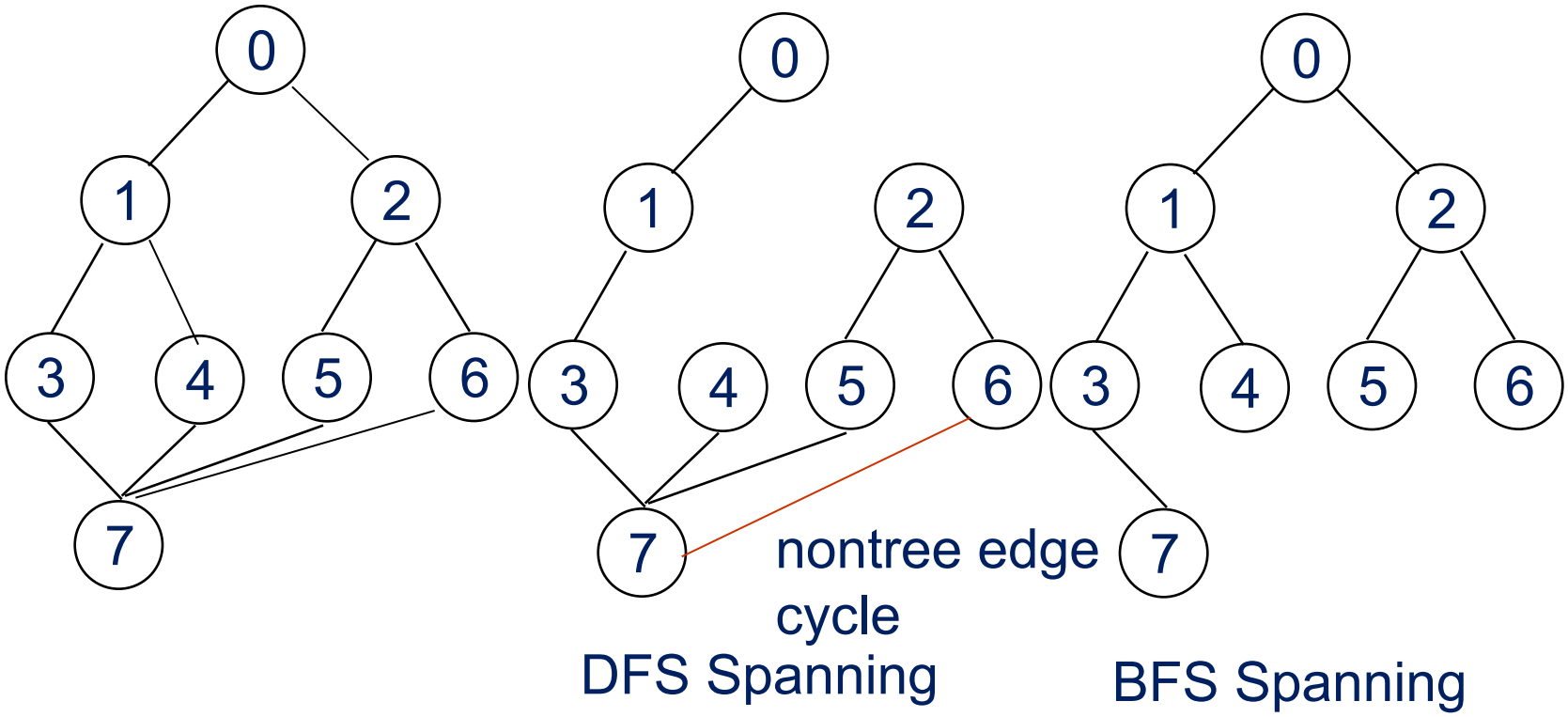


Possible spanning trees

Spanning Trees

- Either dfs or bfs can be used to create a spanning tree
  - When dfs is used, the resulting spanning tree is known as a depth first spanning tree
  - When bfs is used, the resulting spanning tree is known as a breadth first spanning tree
- While adding a nontree edge into any spanning tree, this will create a cycle

DFS VS BFS Spanning Tree

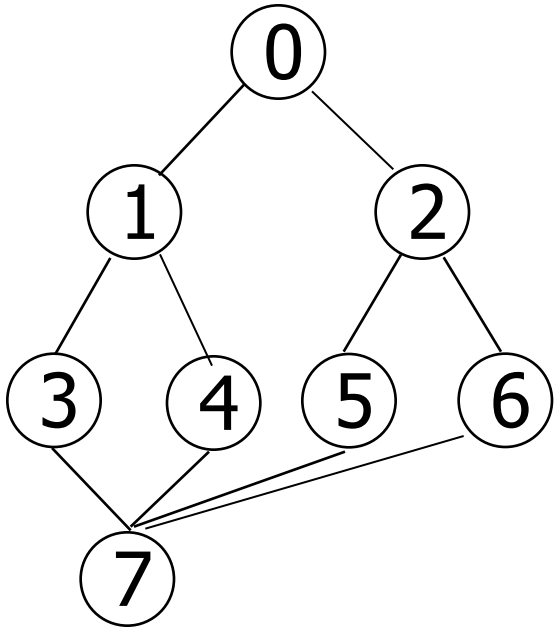


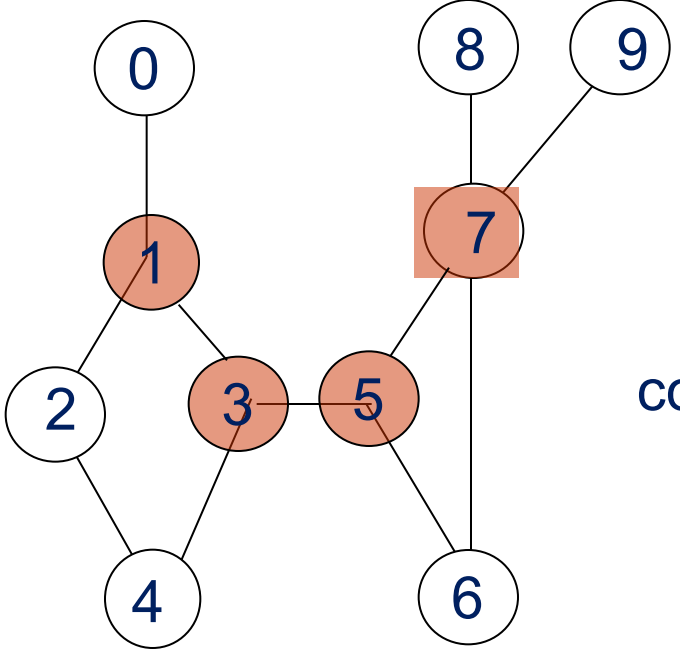
A spanning tree is a minimal subgraph, G', of G such that V(G')=V(G) and G' is connected.

Any connected graph with n vertices must have at least n-1 edges.

A biconnected graph is a connected graph that has no articulation points.

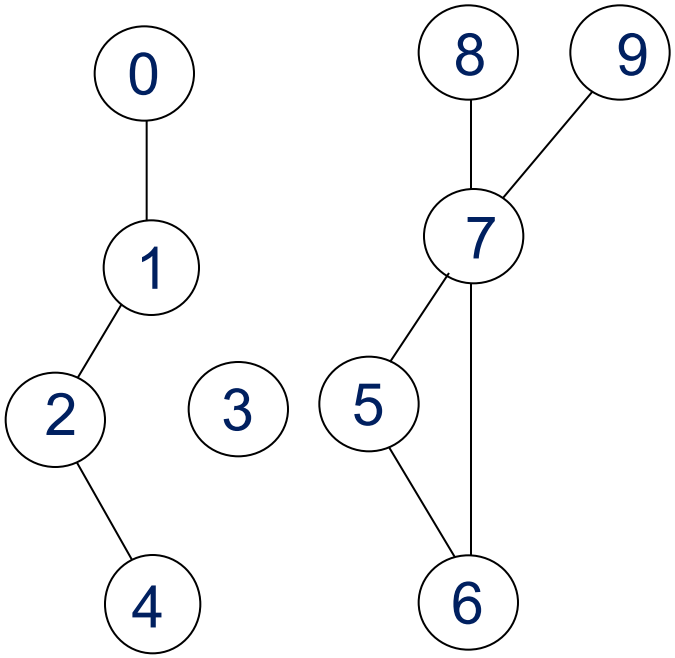
biconnected graph



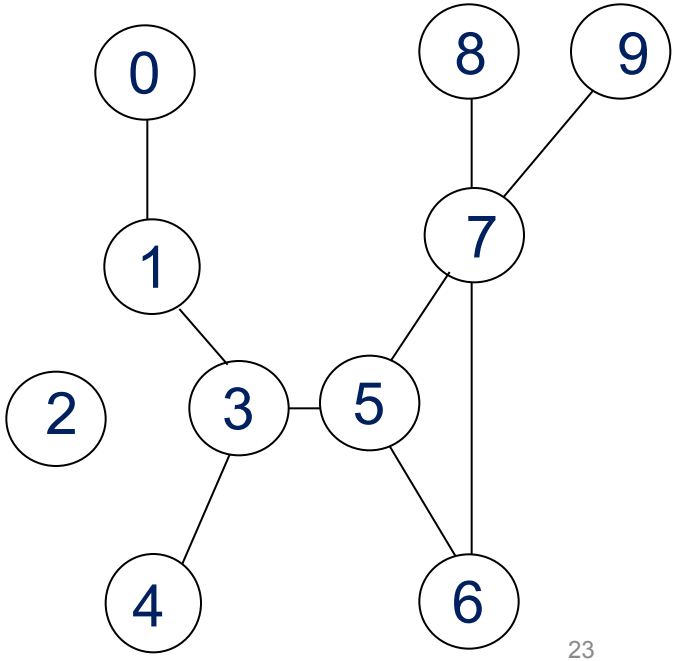


connected graph

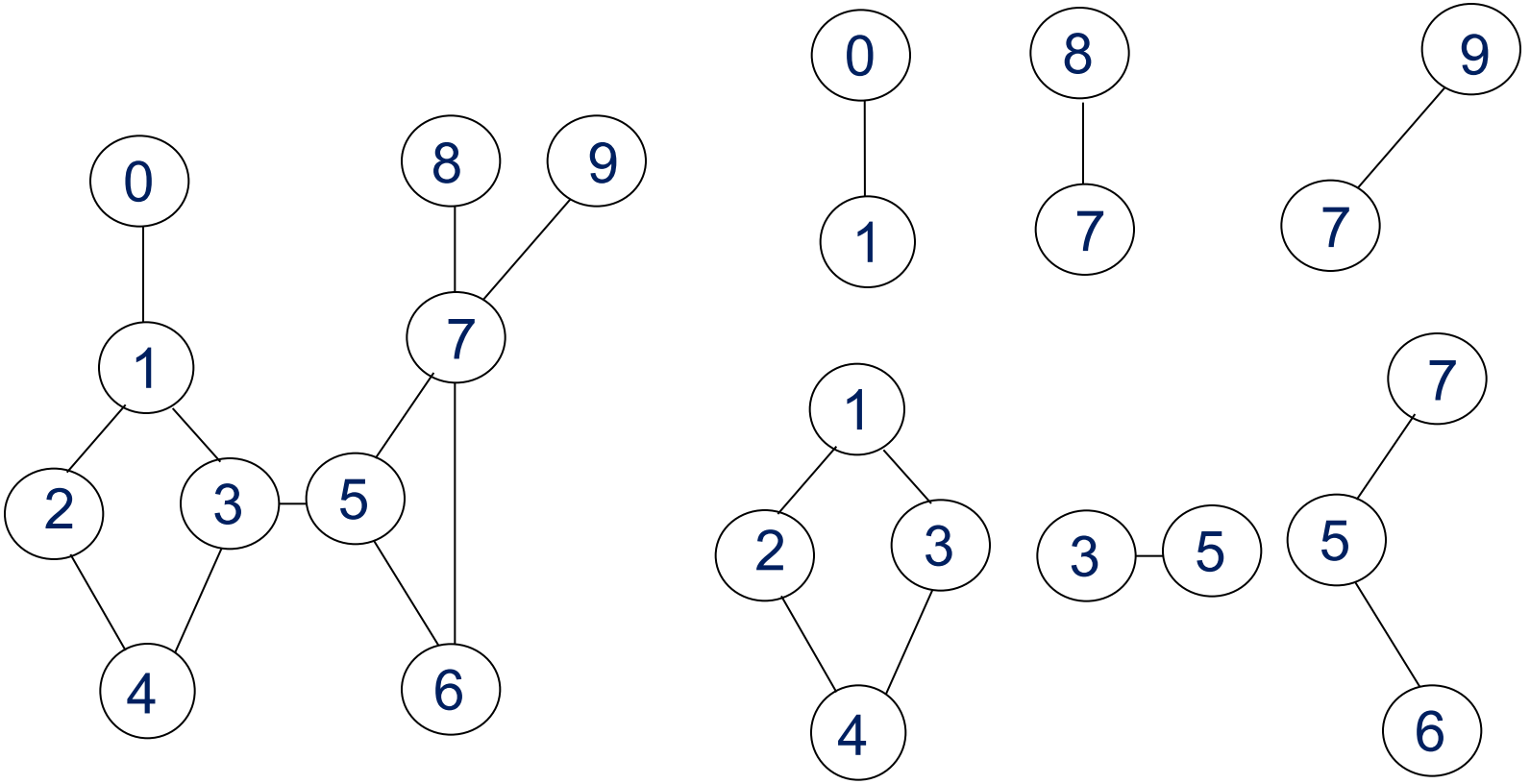
two connected components



one connected graph

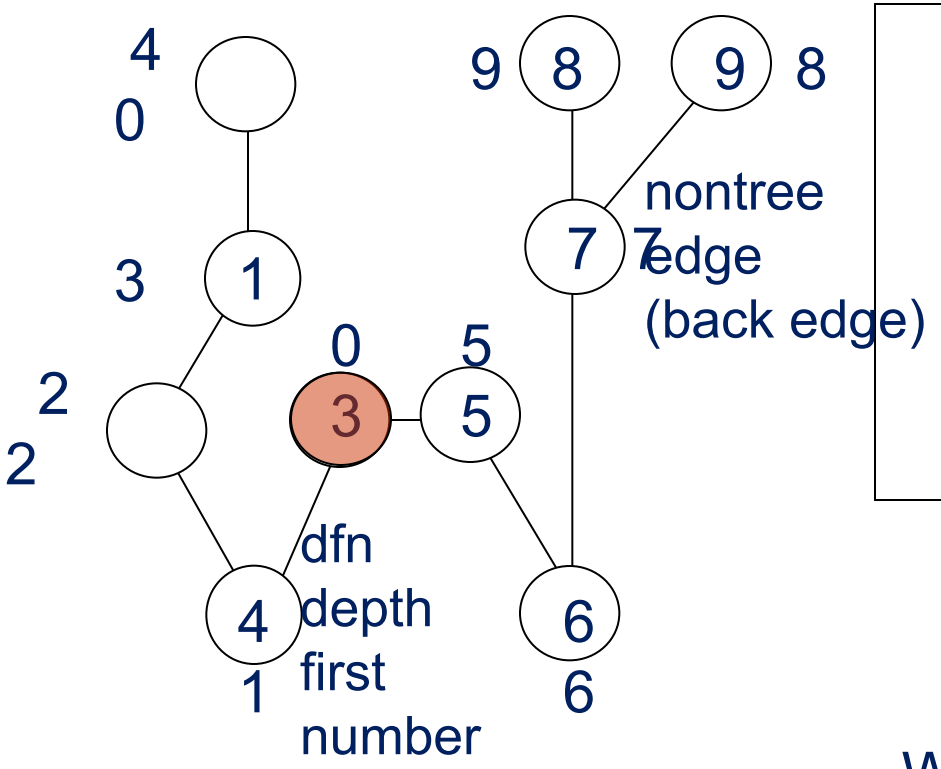


biconnected component: a maximal connected subgraph H
(no subgraph that is both biconnected and properly contains H)

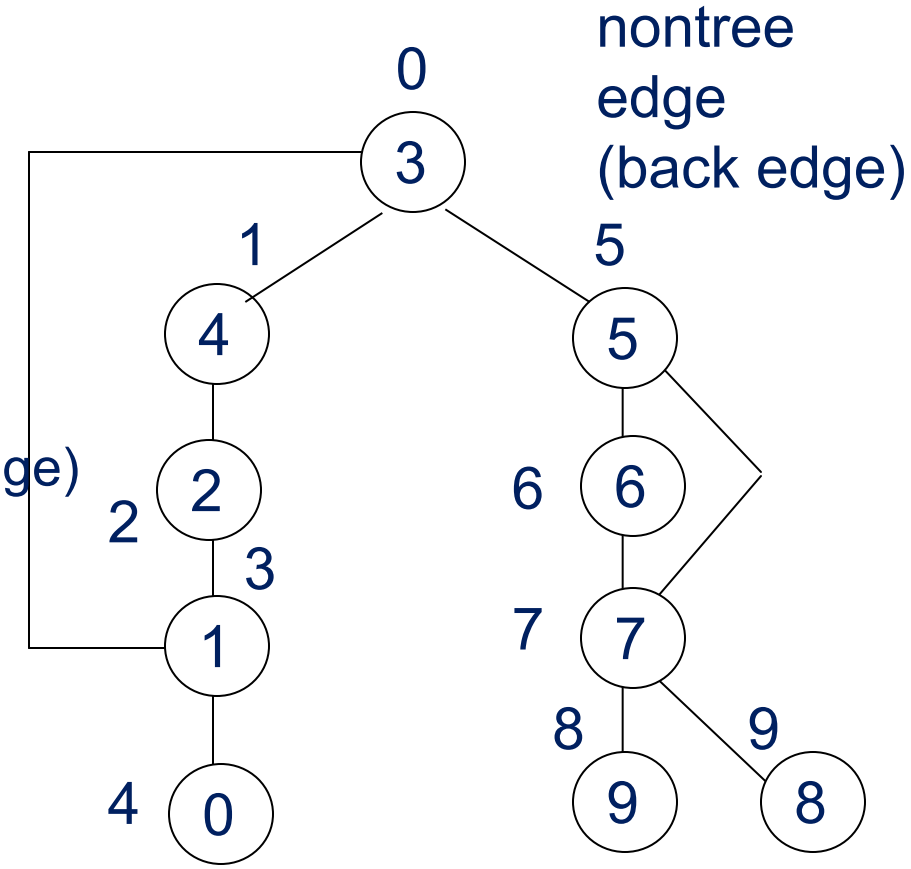


biconnected components

Find biconnected component of a connected undirected graph by depth first spanning tree



(a) depth first spanning tree



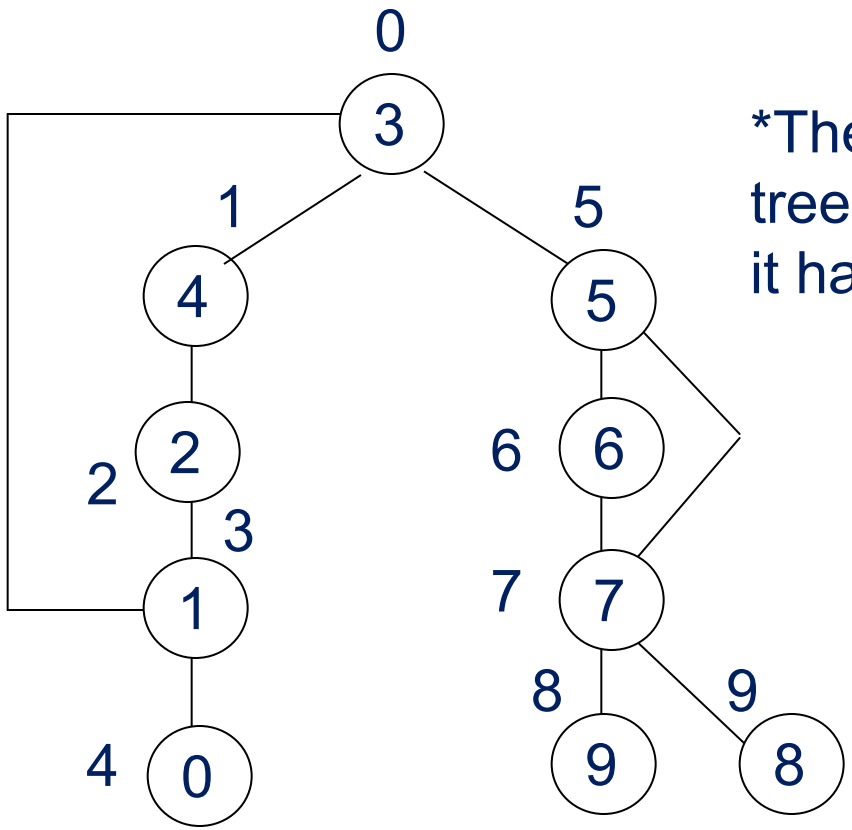
Why is cross edge impossible?

(b)

If u is an ancestor of v then dfn(u) < dfn(v).

dfn and *low* values for *dfs* spanning tree with *root* =3

Vertax	0	1	2	3	4	5	6	7	8	9
dfn	4	3	2	0	1	5	6	7	9	8
low	4	0	0	0	0	5	5	5	9	8



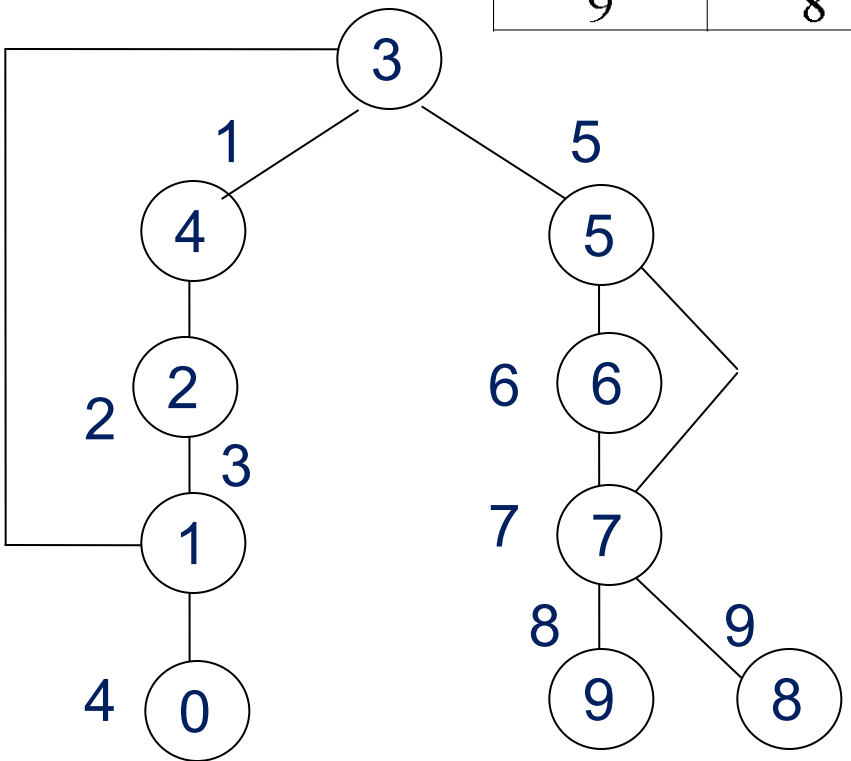
*The root of a depth first spanning tree is an articulation point iff it has at least two children.

*Any other vertex u is an articulation point iff it has at least one child w such that we cannot reach an ancestor of u using a path that consists of (1) only w (2) descendants of w (3) single back edge.

low(u)=min{dfn(u),
min{low(w)|w is a child of u},
min{dfn(w)|(u,w) is a back edge}

u: articulation point
low(child) ≥ dfn(u)

vertex	dfn	low	child	low_child	low:dfn
0	4	4 (4,n,n)	null	null	null:4
1	3	0 (3,4,0)	0	4	4 ≥ 3 ●
2	2	0 (2,0,n)	1	0	0 < 2
3	0	0 (0,0,n)	4,5	0,5	0,5 ≥ 0 ●
4	1	0 (1,0,n)	2	0	0 < 1
5	5	5 (5,5,n)	6	5	5 ≥ 5 ●
6	6	5 (6,5,n)	7	5	5 < 6
7	7	5 (7,8,5)	8,9	9,8	9,8 ≥ 7 ●
8	9	9 (9,n,n)	null	null	null, 9
9	8	8 (8,n,n)	null	null	null, 8



Initializaiton of *dfn* and *low*

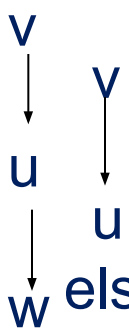
```
void init(void)
 {
   int i;
   for (i = 0; i < n; i++) {
        visited[i] = FALSE;
        dfn[i] = low[i] = -1;
        }
        num = 0;
  }
```

Determining *dfn* and *low*

```
void dfnlow(int u, int v)
{
/* compute dfn and low while performing a dfs search
   beginning at vertex u, v is the parent of u (if any) */
    node_pointer ptr;
    int w;
    dfn[u] = low[u] = num++;
    for (ptr = graph[u]; ptr; ptr = ptr ->link) {
        w = ptr ->vertex;
        if (dfn[w] < 0) { /*w is an unvisited vertex */
            dfnlow(w, u);
            low[u] = MIN2(low[u], low[w]);
        }
        else if (w != v)
            low[u] =MIN2(low[u], dfn[w] );
    }
}
```

Initial call: dfn(x,-1)

low[u]=min{dfn(u), …}



low[u]=min{…, min{low(w)|w is a child of u}, …}

dfn[w]≠0 is not the first time, indicating back edge

low[u]=min{…,…,min{dfn(w)|(u,w) is a back edge}

Biconnected components of a graph

```
void bicon(int u, int v)
{
/* compute dfn and low, and output the edges of G by their
    biconnected  components , v is the parent ( if any) of the u
   (if any)  in the resulting spanning tree. It is assumed that all
  entries  of dfn[ ] have been initialized to -1, num has been
  initialized to 0,  and the stack has been set to empty */
    node_pointer ptr;
    int w, x, y;                                    low[u]=min{dfn(u), …}
    dfn[u] = low[u] = num ++;
    for (ptr = graph[u]; ptr; ptr = ptr->link) {
       w = ptr ->vertex;
       if ( v != w && dfn[w] < dfn[u] )
           add(&top, u, w);        /* add edge to stack */
```

(1) dfn[w]=-1 first
(2) dfn[w]!=-1 is not the first time, by back edge

low[u]=min{…, min{low(w)|w is a child of u}, …}

```
if(dfn[w] < 0) {/* w has not been visited */
      bicon(w, u);
      low[u] = MIN2(low[u], low[w]);
      if (low[w] >= dfn[u] ){ articulation point
          printf("New biconnected component: ");
          do { /* delete edge from stack */
               delete(&top, &x, &y);
               printf(" <%d, %d>" , x, y);
           }  while (!(( x = = u) && (y = = w)));
           printf("\n");
        }
     }
     else if (w != v)  low[u] = MIN2(low[u], dfn[w]);
  }
}
```

low[u]=min{…, …, min{dfn(w)|(u,w) is a back edge}}

Minimum Spanning Tree (MST)

Minimum Cost Spanning Tree

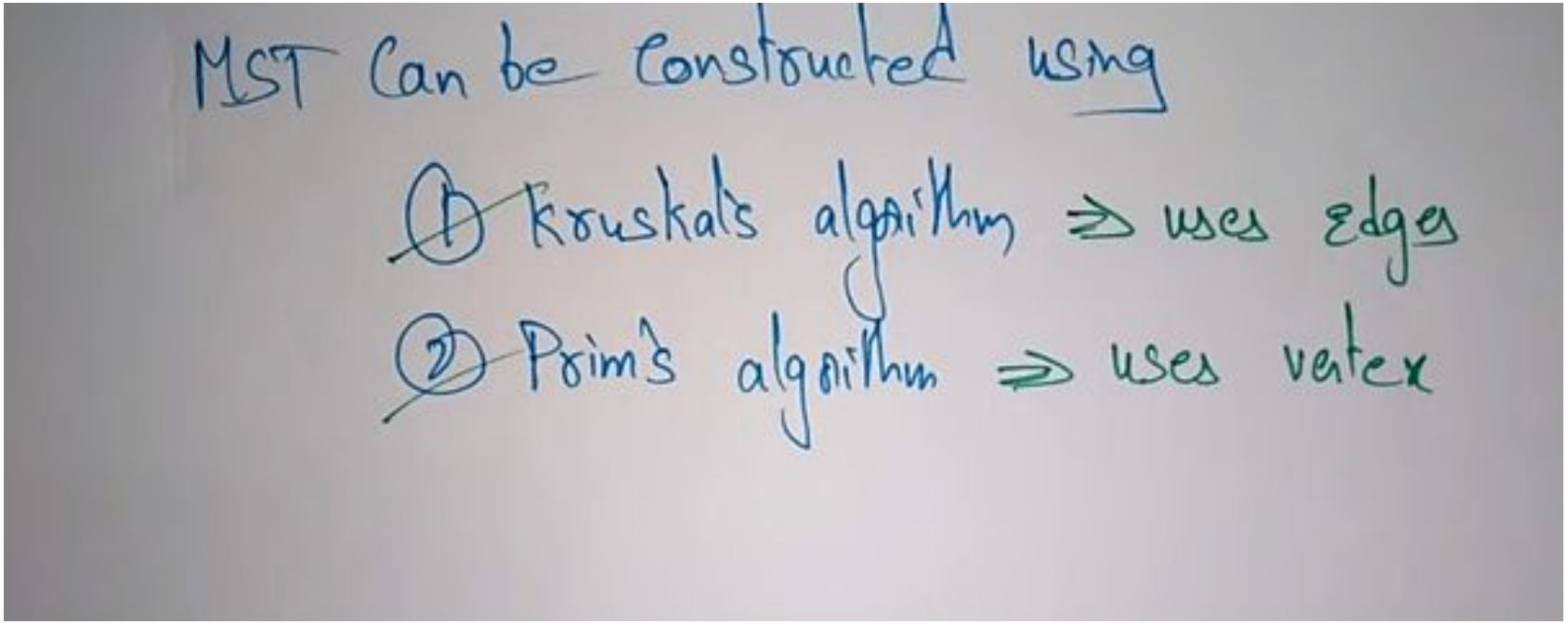
- The cost of a spanning tree of a weighted undirected graph is the sum of the costs of the edges in the spanning tree
- A minimum cost spanning tree is a spanning tree of least cost
- Two different algorithms can be used
  - Kruskal
  - Prim

Select n-1 edges from a weighted graph of n vertices with minimum cost.

Greedy Strategy

- An optimal solution is constructed in stages
- At each stage, the best decision is made at this time
- Since this decision cannot be changed later, we make sure that the decision will result in a feasible solution
- Typically, the selection of an item at each stage is based on a least cost or a highest profit criterion

MST

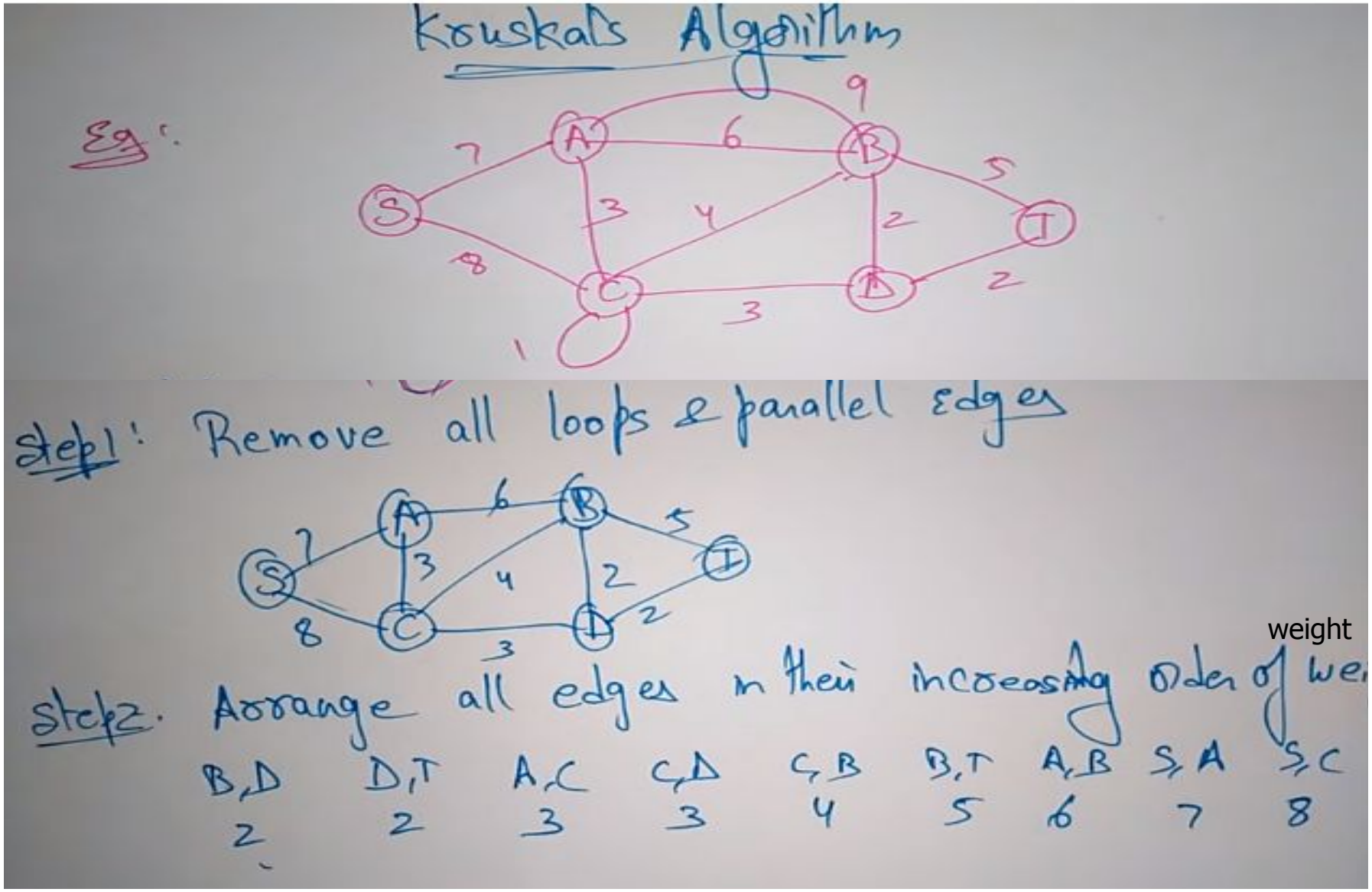

MST Can be Constructed using
1 Kruskal's algorithm ⇒ uses edges
2 Prim's algorithm ⇒ uses vertex

Kruskal's Idea

- Build a minimum cost spanning tree T by adding edges to T one at a time
- Select the edges for inclusion in T in nondecreasing order of the cost
- An edge is added to T if it does not form a cycle
- Since G is connected and has n > 0 vertices, exactly n-1 edges will be selected

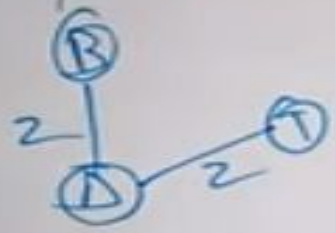
https://www.youtube.com/watch?v=xn11tv6B4zw

Examples for Kruskal's Algorithm

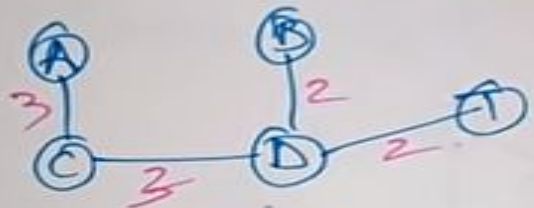


Examples for Kruskal's Algorithm

Step 3: Add the edge which has least weightage



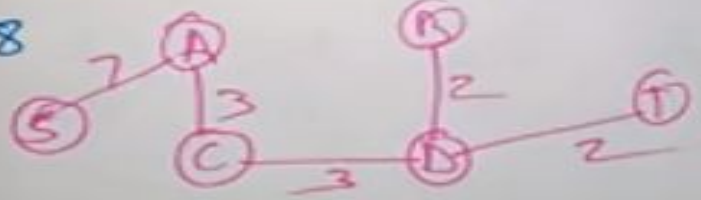
Next Cost is 3, the edges are AC & CD



Next cost is 4 (discard) b/s to avoid circuit

ignore (6 & 5) also

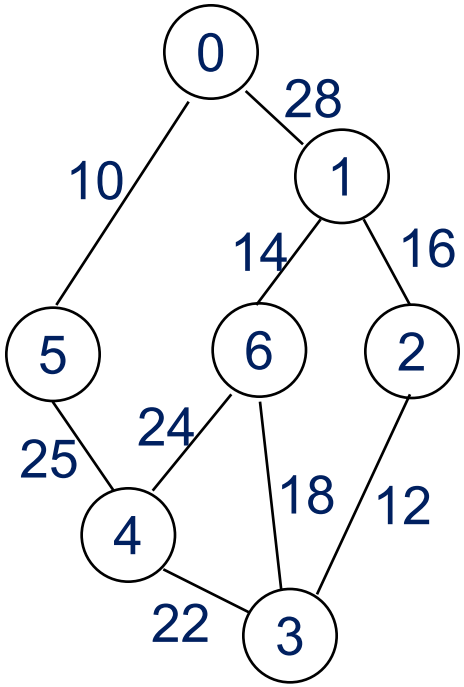
Next Cost is 7, 8

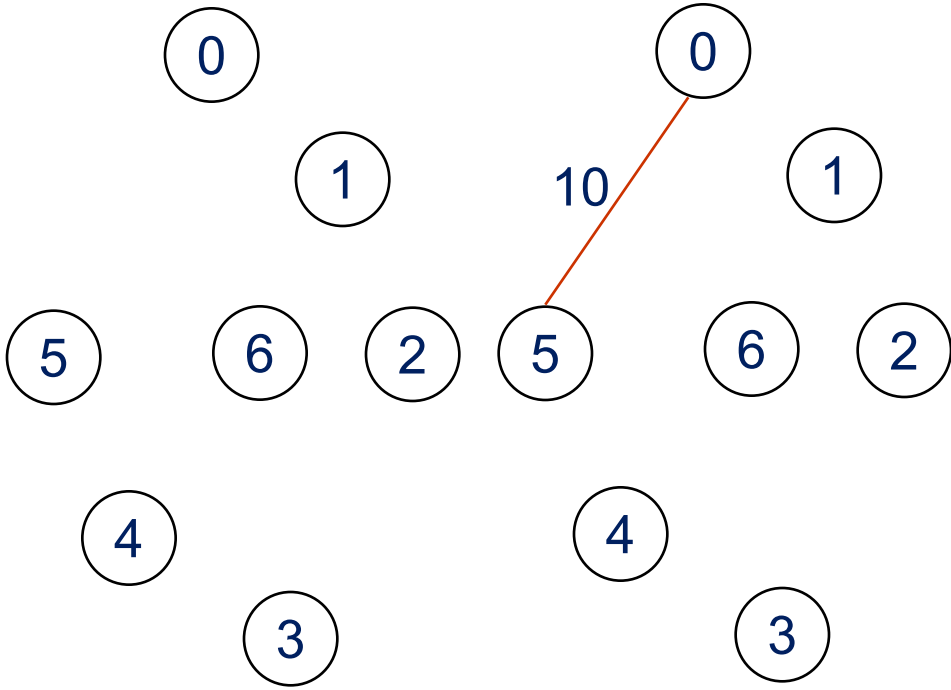


MST is Com

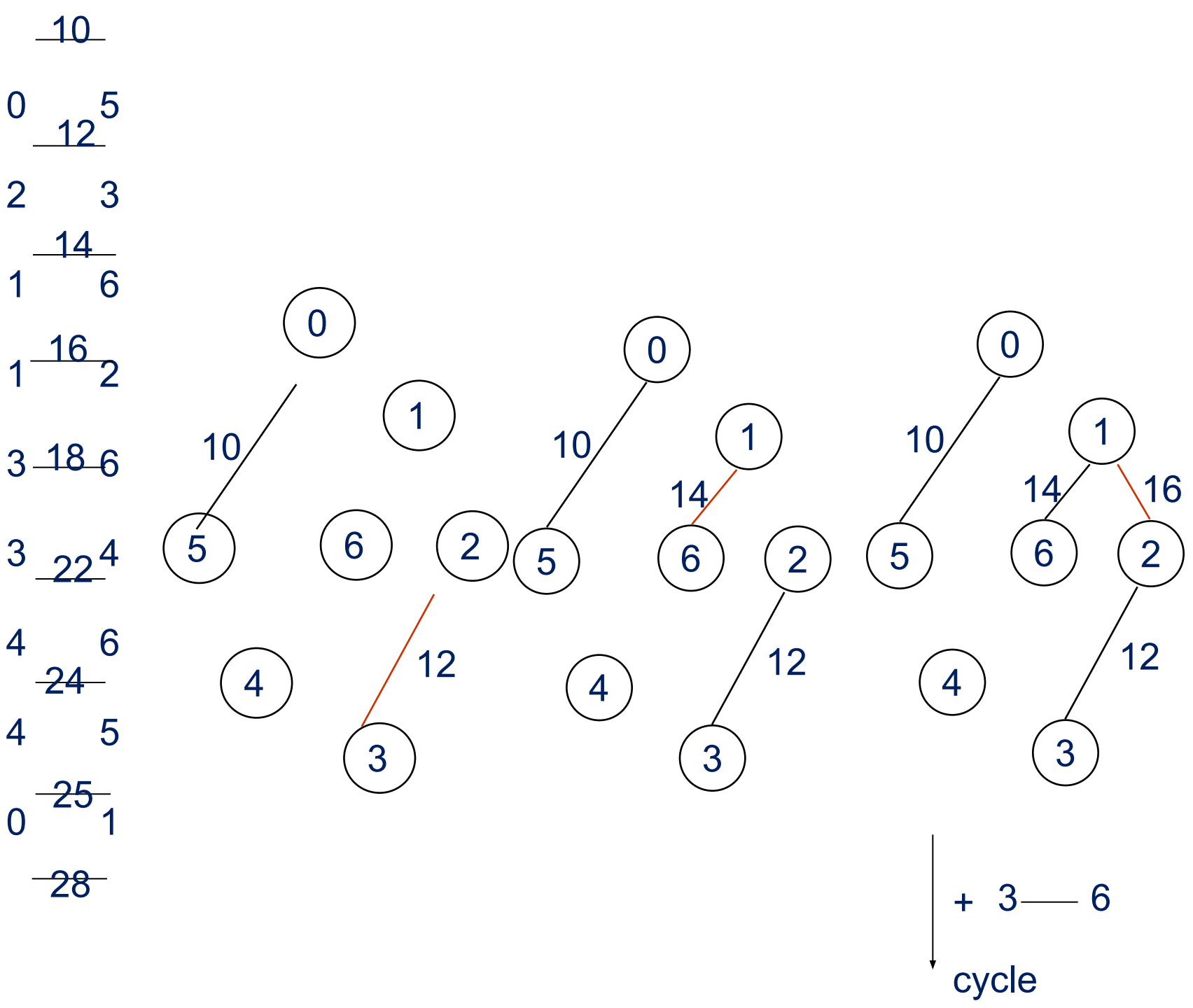
Examples for Kruskal's Algorithm

0	5	10
2	3	12
1	6	~~14~~
1	2	16
3	6	~~18~~
3	4	~~22~~
4	6	~~24~~
4	5	25
0	1	28





6/9


10
0 5
12
2 3
14
1 6
16
1 2
18
3 6
3 22 4
24
4 6
4 5
25
0 1
28
0
10
5
1
6
2
12
3
4
0
10
5
1
14
6
2
12
3
4
0
10
5
1
14
16
6
2
12
3
4
+ 3 6
cycle

1
0

0 1 5
2

2 1 3
4

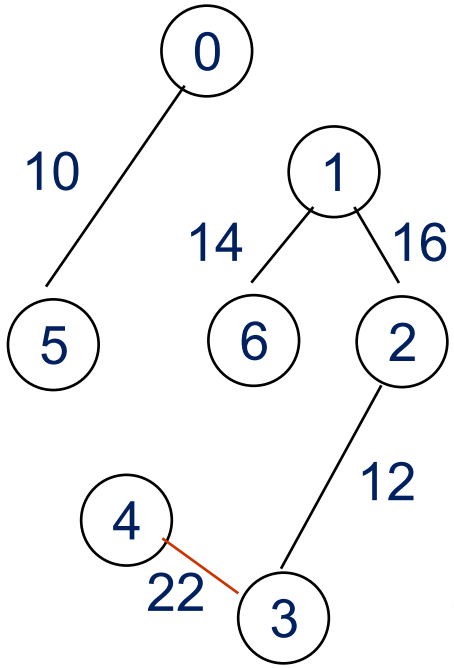
1 6
1
1 6 2

1
3 8 6

3 2 4
2

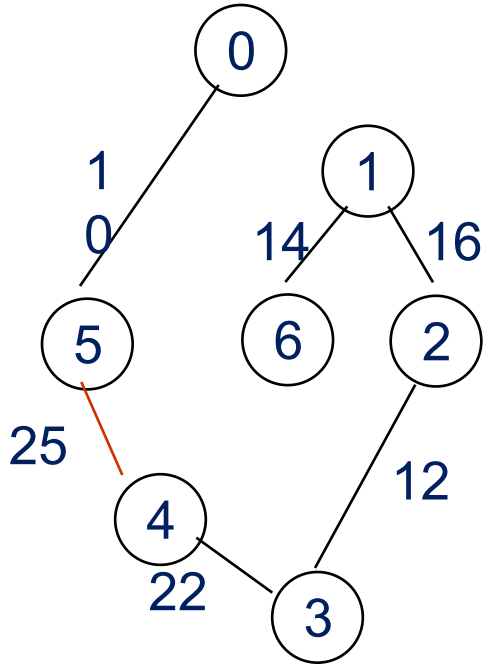
4 2 6
4

4 5
2
0 5 1
2
8



+

4
6
cycl
e



cost = 10
+25+22+12+16+14

Kruskal's Algorithm

Goal: Take out N-1 edges

```
T= {};
while (T contains less than n-1 edges
        && E is not empty) {
choose a least cost edge (v,w) from E;
delete (v,w) from E;
if ((v,w) does not create a cycle in T)
    add (v,w) to T
 else discard (v,w);
}
if (T contains fewer than n-1 edges)
  printf("No spanning tree\n");
```

min heap construction time
O(e)
choose and delete O(log e)

find find & union O(log e)

{0,5}, {1,2,3,6}, {4} + edge(3,6) X + edge(3,4) --> {0,5},{1,2,3,4,6}

O(e log e)

Prim's Algorithm

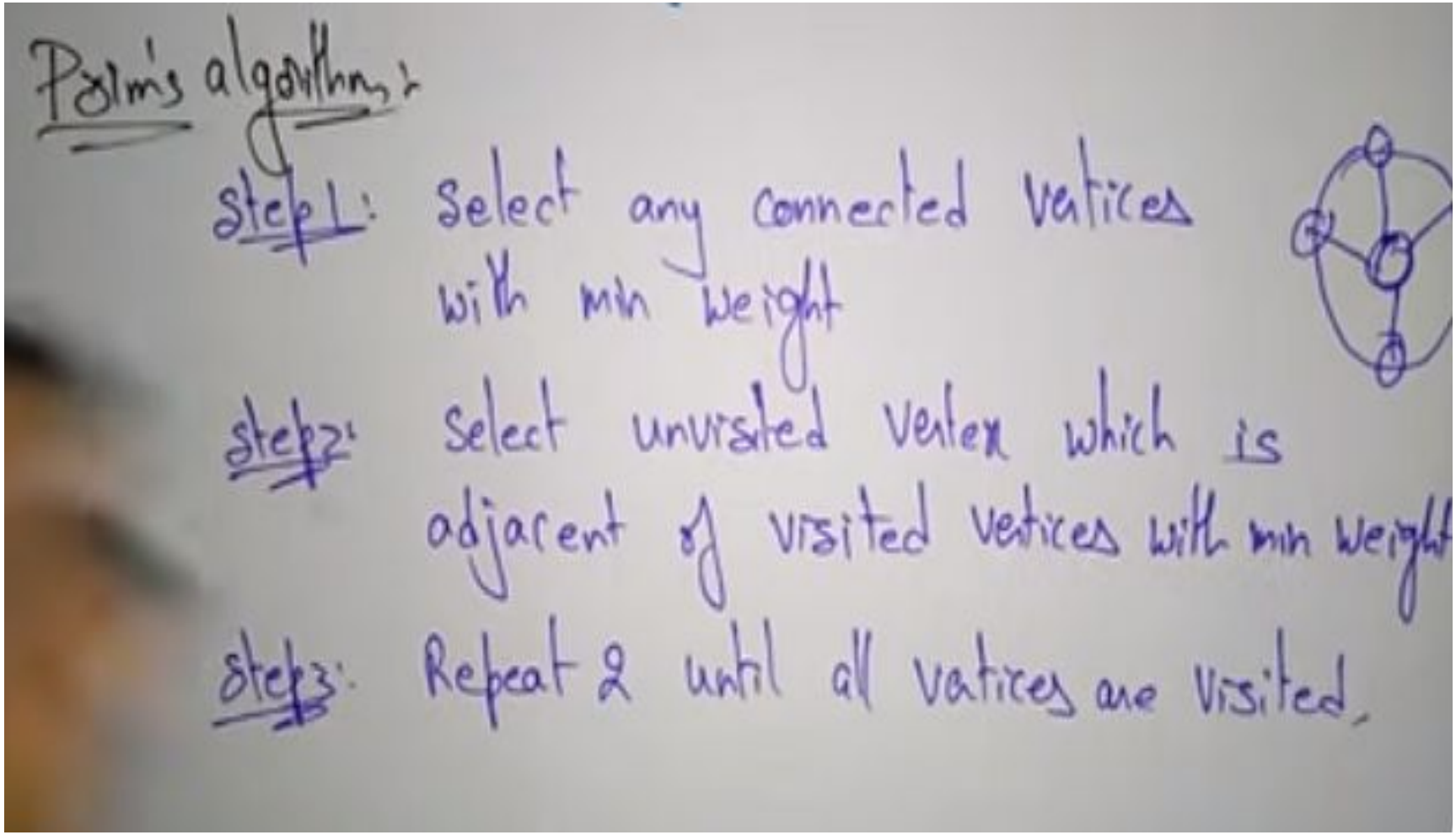
(tree all the time vs. forest)

```
T={};
TV={0};
while (T contains fewer than n-1 edges)
{
  let (u,v) be a least cost edge such
    that u ∈ TV and  v ∉ TV
  if (there is no such edge ) break;
  add v to TV;
  add (u,v) to T;
}
if (T contains fewer than n-1 edges)
  printf("No spanning tree\n");
```

https://www.youtube.com/watch?v=FVoXffphlCg

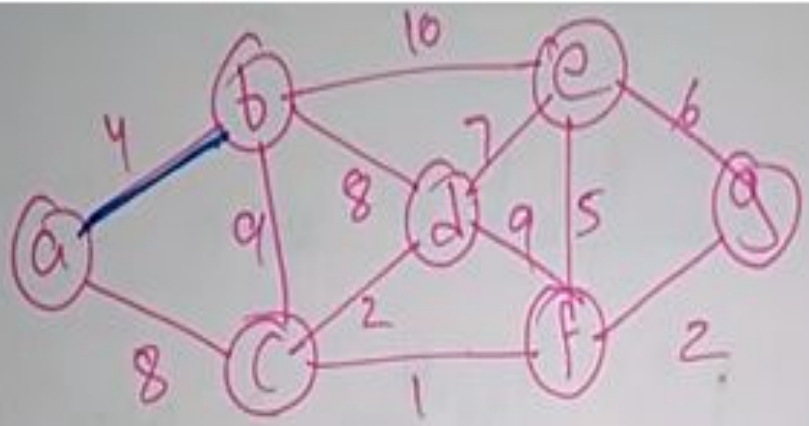
Difference: https://www.youtube.com/watch?v=Yu0Ply5_-2o

Examples for Prim's Algorithm



Examples for Prim's Algorithm

Eg:



Connected graph

Step 1:

(a)

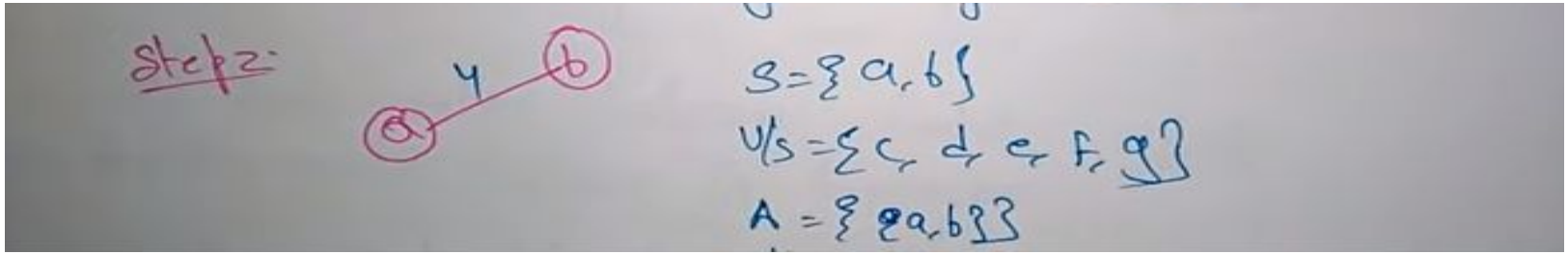
$S = \{a\}$

$V_S = \{b, c, d, e, f, g\}$

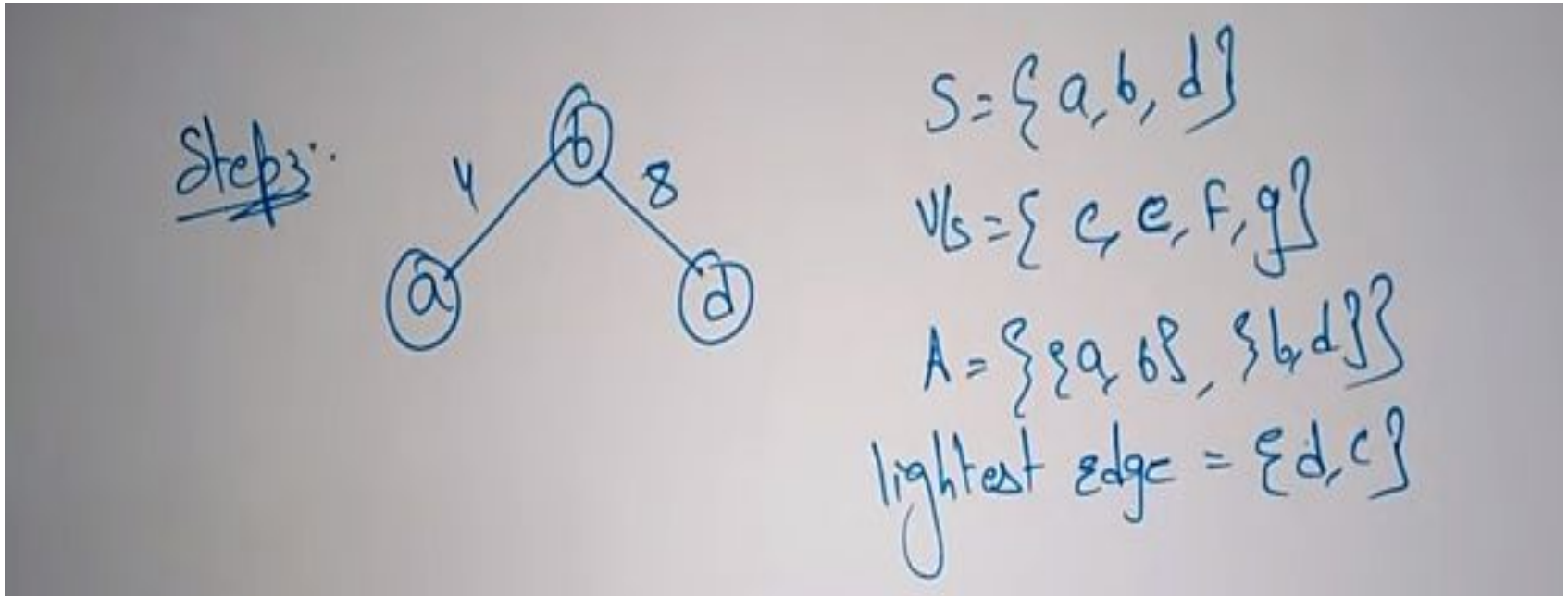
$A = \{\}$

lightest edges $= \{a, b\}$

Examples for Prim's Algorithm



Lightest edges={(b,d)}



Examples for Prim's Algorithm

Step 4.

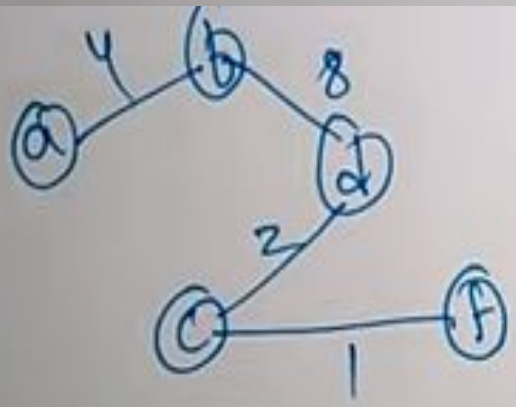
$S = \{a, b, c, d\}$

$V - S = \{e, f, g\}$

$A = \{\{a, b\}, \{b, d\}, \{d, c\}\}$

lightest edge $= \{c, f\}$

Steps



$S = \{a, b, c, d, f\}$

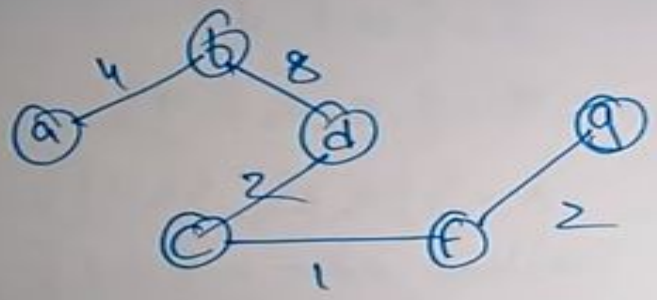
$V - S = \{e, g\}$

$A = \{\{a, b\}, \{b, d\}, \{d, c\}, \{c, f\}\}$

lightest edge $= \{f, g\}$

Examples for Prim's Algorithm

Step 6:



$S = \{a, b, c, d, f, g\}$

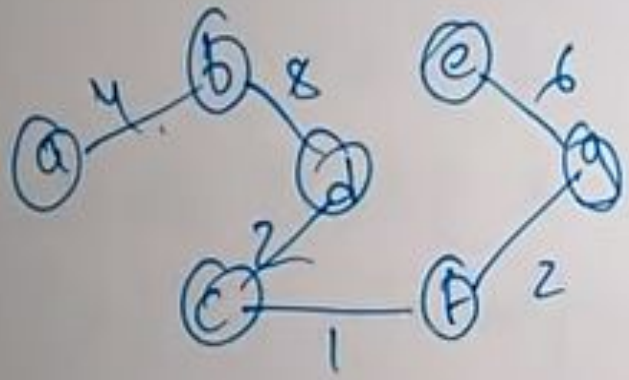
$V/S = \{e\}$

{f,g}}

$A = \{\{a,b\}\{b,d\}, \{d,c\}, \{c,f\}$

lightest edge = {g, e}

Step 7:



$S = \{a, b, c, d, e, f, g\}$

$V/S = \{\}$

{f,g}}

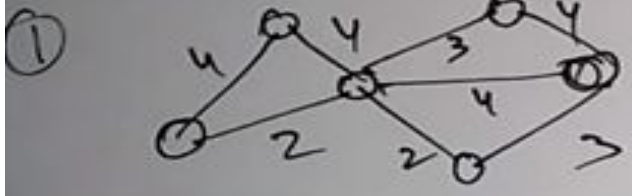
$A = \{\{a,b\}\{b,d\}\{d,c\}\{c,f\}\{f$

$\{g,e\}$

MST Completed

Difference betwee s' Kruskals and Prims:

Kruskals Algorithm

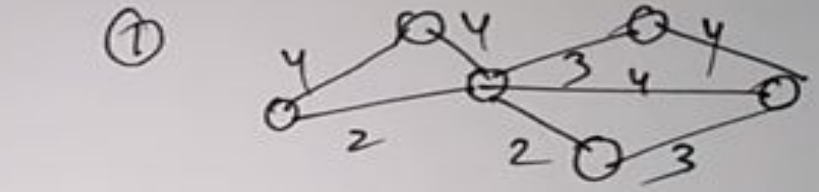


Choose the shortest edge

Choose the next shortest edge & add it

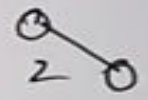
Choose the next shortest which wouldn't create a cycle

Prims Algorithm



Choose a vertex

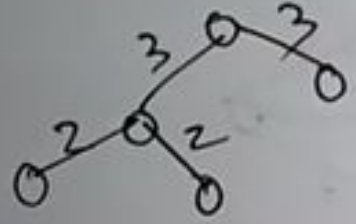
Choose a shortest edge from this vertex



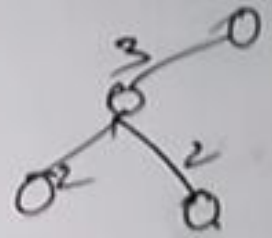
Choose the nearest vertex not Yet in solution.

Difference betwee s' Kruskals and Prims:

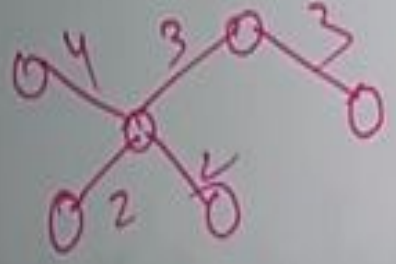
Choose the next shotest
which wouldnt create a cycle



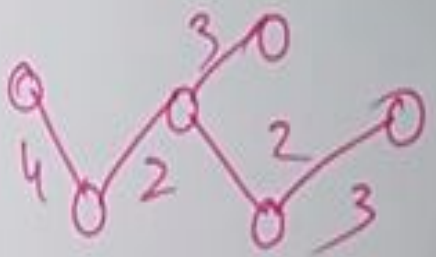
⑤ Choose the next neanest Ve
Vertice



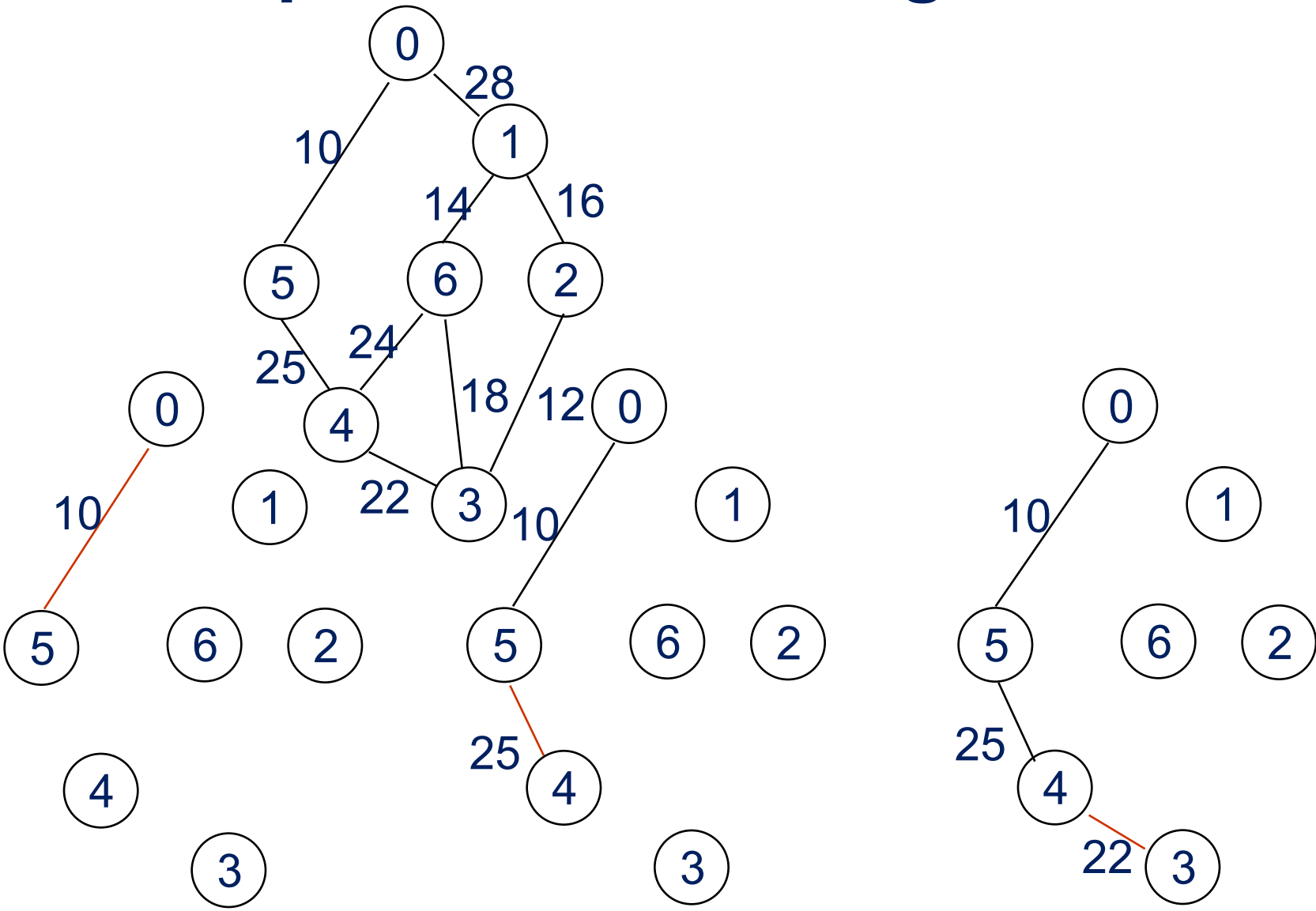
Repeat until you have MST

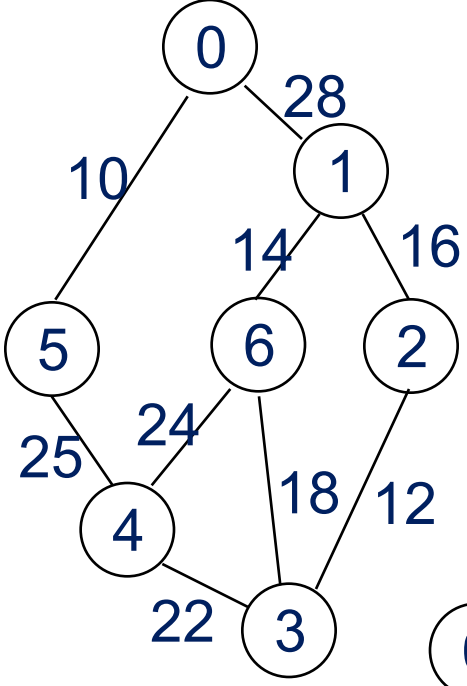


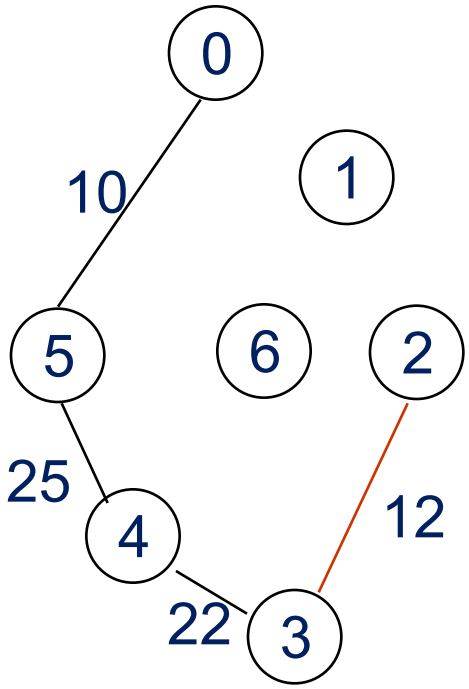
⑥ Repeat until u have a MST

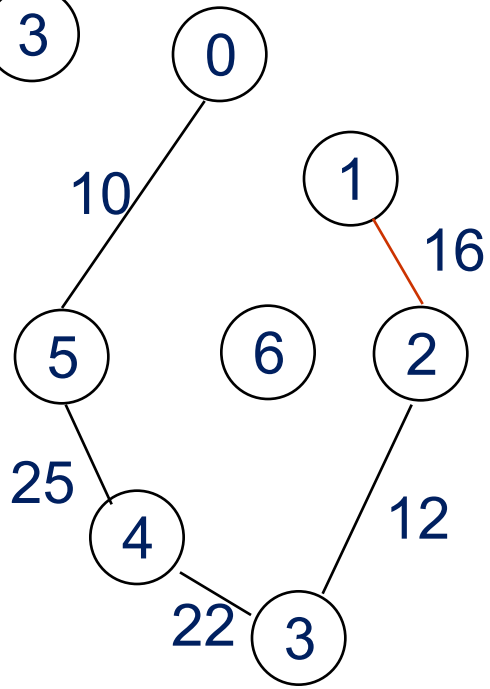


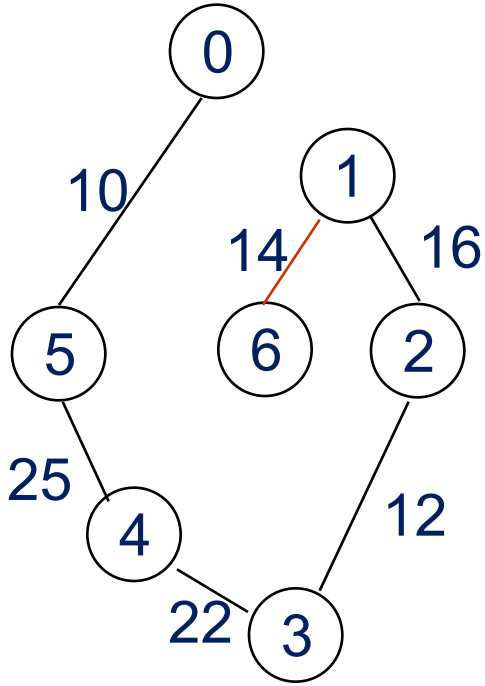
Examples for Prim's Algorithm




0
28
10
1
14
16
5
6
2
24
25
18
12
4
22
3


0
10
1
5
6
2
25
12
4
22
3

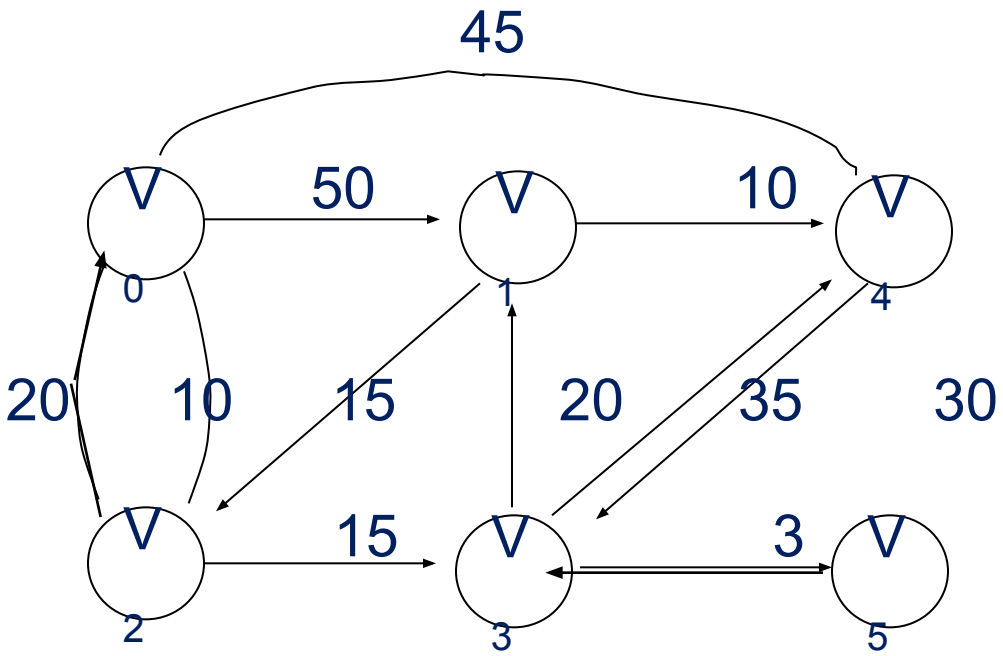

0
10
1
16
5
6
2
25
12
4
22
3


0
10
1
14
16
5
6
2
25
12
4
22
3

Single Source All Destinations

Determine the shortest paths from v0 to all the remaining vertices.

Graph and shortest paths from v_0

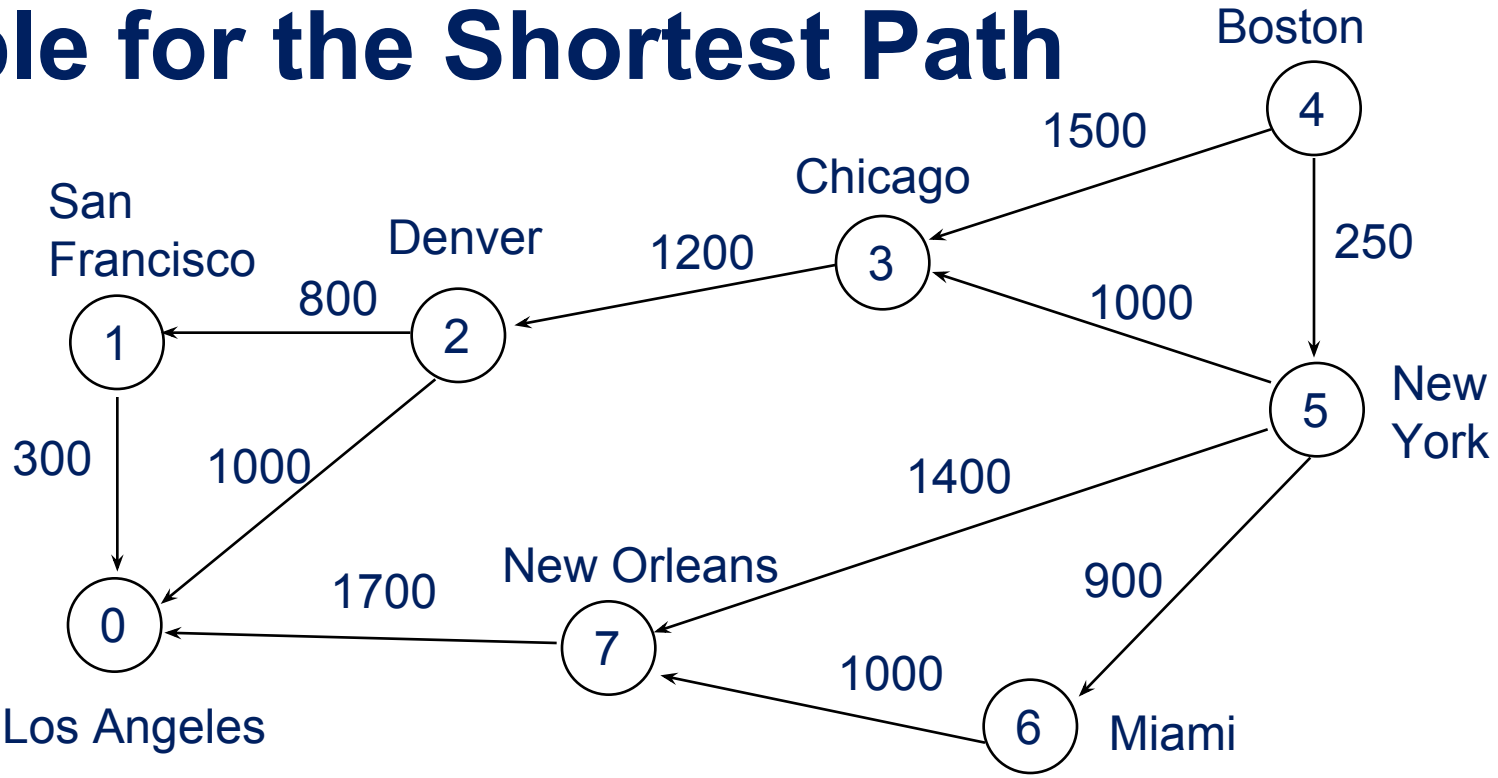


(a)

path	length
1) v0 v2	10
2) v0 v2 v3	25
3) v0 v2 v3 v1	45
4) v0 v4	45

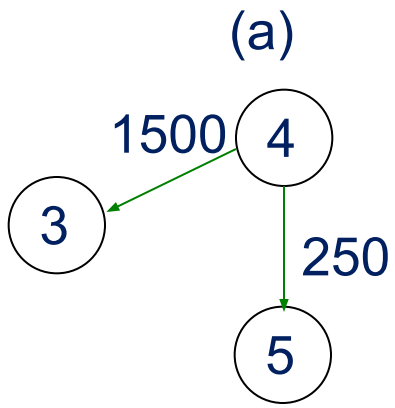
(b)

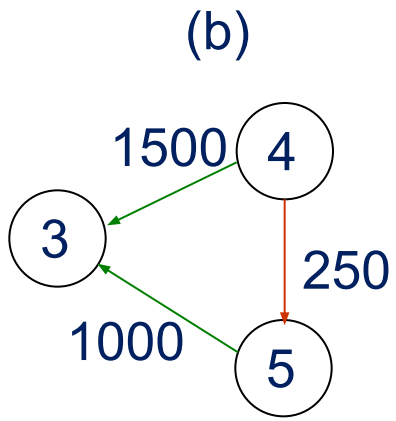
Example for the Shortest Path

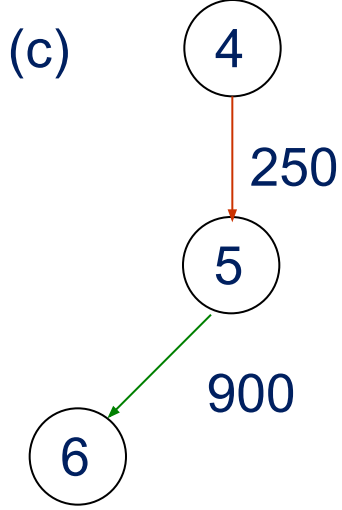


	0	1	2	3	4	5	6	7
0	0							
1	300	0						
2	1000		800	0				
3			1200		0			
4				1500		0	250	
5				1000			0	900 1400
6							0	1000
7	1700							0

Cost adjacency matrix



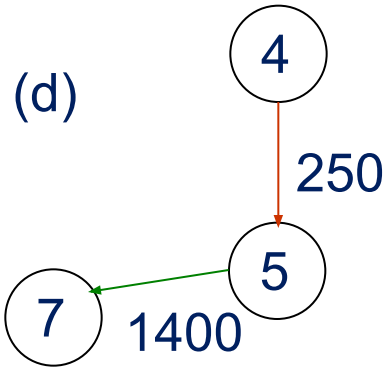


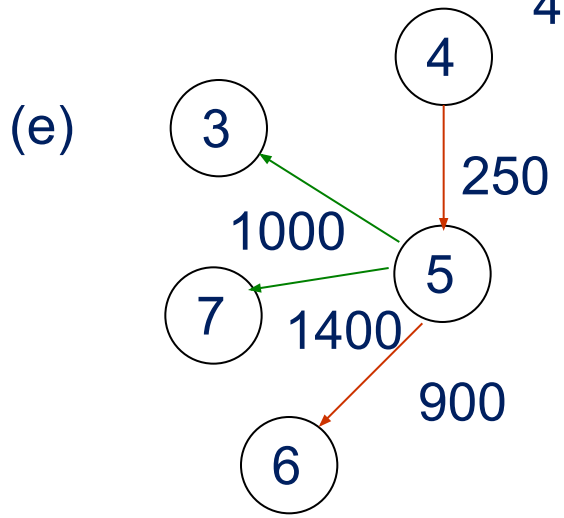


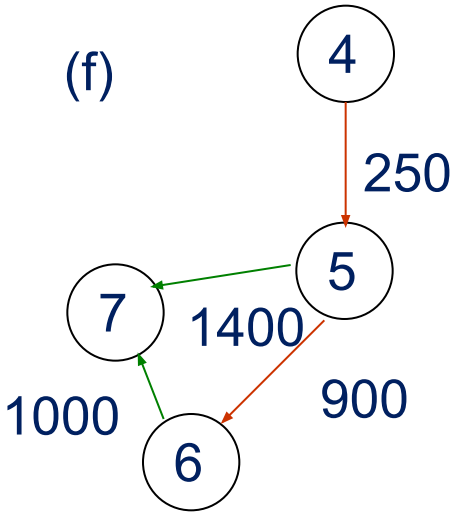
Pick 5

4 to 3 changed from 1500 to 1250

4 to 6 changed from □ to 1150





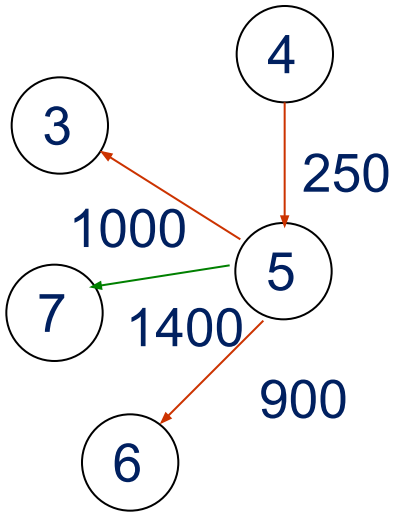


Pick 6

4 to 7 changed from □ to 1650

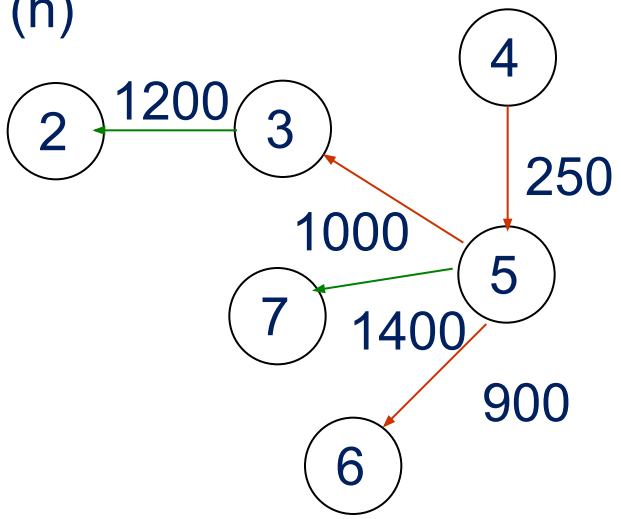
4-5-6-7 is longer than 4-5-7

(g)



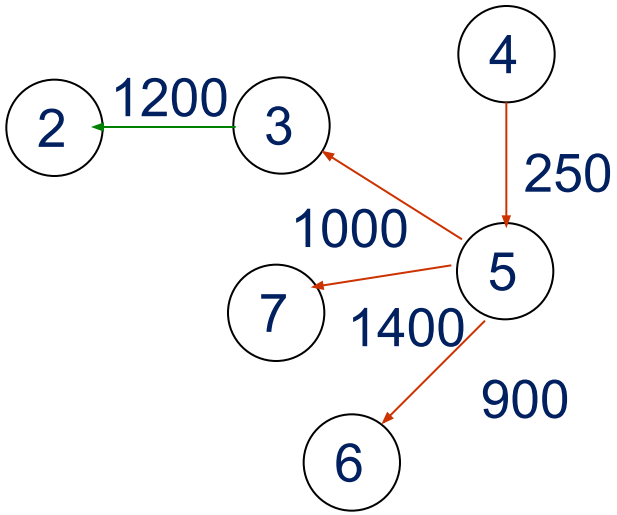
Pick 3

(h)



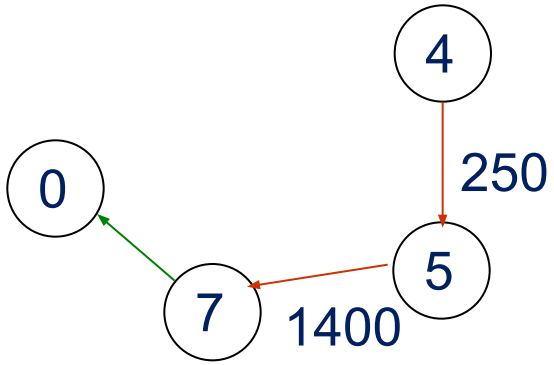
4 to 2 changed from □ to 2450

(i)



Pick 7

(j)



4 to 0 changed from □ to 3350

Example for the Shortest Path (Continued)

Iteration	S	Vertex Selected	LA [0]	SF [1]	DEN [2]	CHI [3]	BO [4]	NY [5]	MIA [6]	NO
Initial	--	----	+∞	+∞	+∞ (b)	1500	0	250 (c)	+∞ (d)	+∞
1	{4} (a)	5	+∞	+∞	+∞	1250	0	250	1150	1650 (f)
2	{4,5} (e)	6	+∞	+∞ (h)	+∞	1250	0	250	1150	1650
3	{4,5,6} (g)	3	+∞	+∞	2450	1250	0	250	1150	1650
4	{4,5,6,3} (i)	7	(j) 3350	+∞	2450	1250	0	250	1150	1650
5	{4,5,6,3,7}	2	3350	3250	2450	1250	0	250	1150	1650
6	{4,5,6,3,7,2}	1	3350	3250	2450	1250	0	250	1150	1650
7	{4,5,6,3,7,2,1}									

Data Structure for SSAD

```
#define MAX_VERTICES 6
int cost[][MAX_VERTICES]=
    {{   0,   50,   10, 1000,   45, 1000},
     {1000,    0,   15, 1000,   10, 1000},
     {  20, 1000,    0,   15, 1000, 1000},
     {1000,   20, 1000,    0,   35, 1000},
     {1000, 1000,   30, 1000,    0, 1000},
     {1000, 1000, 1000,    3, 1000,    0}};
int distance[MAX_VERTICES];
short int found{MAX_VERTICES];
int n = MAX_VERTICES;
```

adjacency matrix

Single Source All Destinations

```
void shortestpath(int v, int cost[][MAX_ERXTICES],
    int distance[], int n, short int found[])
{
  int i, u, w;
  for (i=0; i<n; i++) {
    found[i] = FALSE;
    distance[i] = cost[v][i];
  }
  found[v] = TRUE;
  distance[v] = 0;
```

O(n
)

```
  for (i=0; i<n-2; i++) {determine n-1 paths from v
    u = choose(distance, n, found);
    found[u] = TRUE;
    for (w=0; w<n; w++)
      if (!found[w])
        if (distance[u]+cost[u][w]<distance[w])
          distance[w] = distance[u]+cost[u][w];
  }
}
```

The endpoint w connected to u

$O(n^2)$

All Pairs Shortest Paths

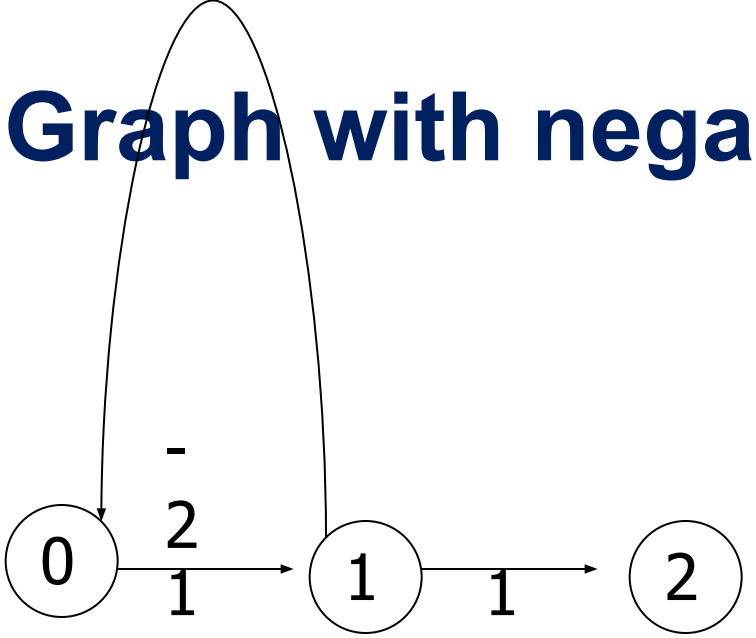
- Find the shortest paths between all pairs of vertices.
- Solution 1
  - Apply shortest path n times with each vertex as source.

    $O(n^3)$
- Solution 2
  - Represent the graph G by its cost adjacency matrix with cost[i][j]
  - If the edge <i,j> is not in G, the cost[i][j] is set to some sufficiently large number
  - A[i][j] is the cost of the shortest path form i to j, using only those intermediate vertices with an index <= k

All Pairs Shortest Paths (Continued)

- The cost of the shortest path from i to j is $A^{n-1}[i][j]$, as no vertex in G has an index greater than n-1
- $A^{-1}[i][j]=cost[i][j]$
- Calculate the A^0, A^1, A^2, ..., A^{n-1} from A^{-1} iteratively
- $A^k[i][j]=\min\{A^{k-1}[i][j], A^{k-1}[i][k]+A^{k-1}[k][j]\}, k>=0$

Graph with negative cycle



(a) Directed graph

$$\begin{bmatrix} 0 & 1 & \infty \\ -2 & 0 & 1 \\ \infty & \infty & 0 \end{bmatrix}$$

(b) A^{-1}

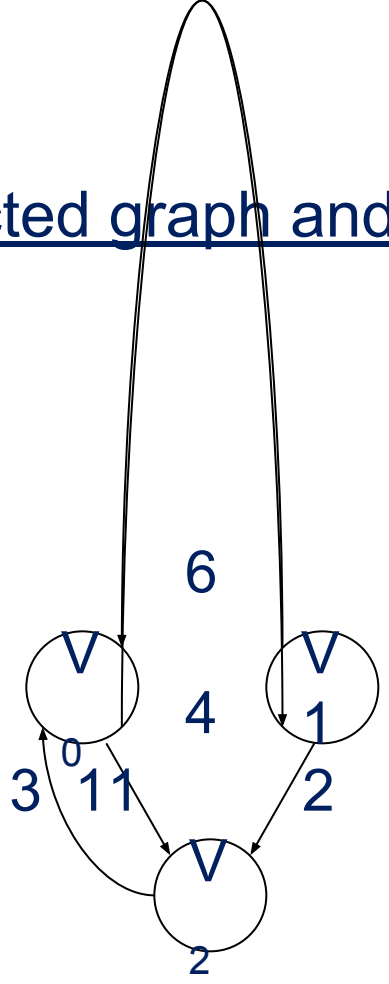
The length of the shortest path from vertex 0 to vertex 2 is $-\infty$.

0, 1, 0, 1,0, 1, …, 0, 1, 2

Algorithm for All Pairs Shortest Paths

```
void allcosts(int cost[][MAX_VERTICES],
        int distance[][MAX_VERTICES], int n)
{
  int i, j, k;
  for (i=0; i<n; i++)
    for (j=0; j<n; j++)
          distance[i][j] = cost[i][j];
  for (k=0; k<n; k++)
    for (i=0; i<n; i++)
      for (j=0; j<n; j++)
        if (distance[i][k]+distance[k][j]
            < distance[i][j])
          distance[i][j]=
              distance[i][k]+distance[k][j];
}
```

Directed graph and its cost matrix



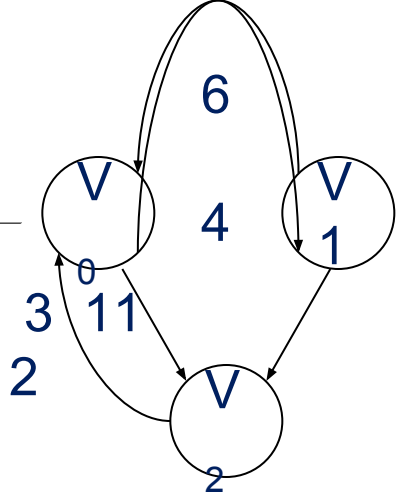
(a)Digraph G

	0	1	2
0	0	4	11
1	6	0	2
2	3	∞	0

(b)Cost adjacency matrix for G

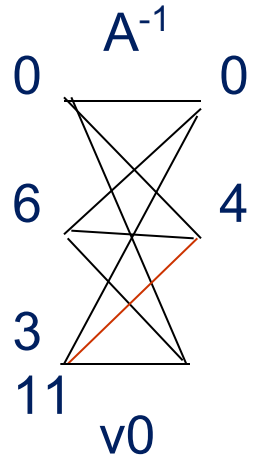
A^{-1}

	0	1	2
0	0	4	11
1	6	0	2
2	3	∞	0



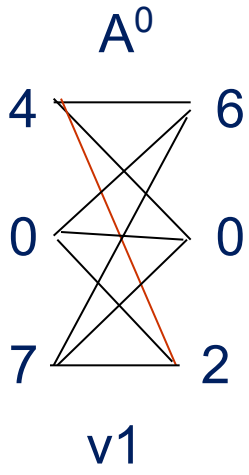
A^{0}

	0	1	2
0	0	4	11
1	6	0	2
2	3	7	0



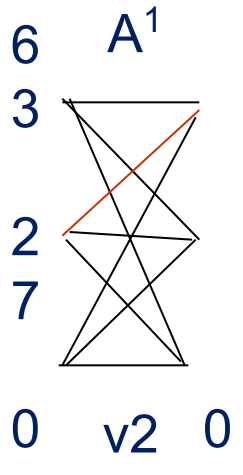
A^{1}

	0	1	2
0	0	4	6
1	6	0	2
2	3	7	0



A^{2}

	0	1	2
0	0	4	6
1	5	0	2
2	3	7	0

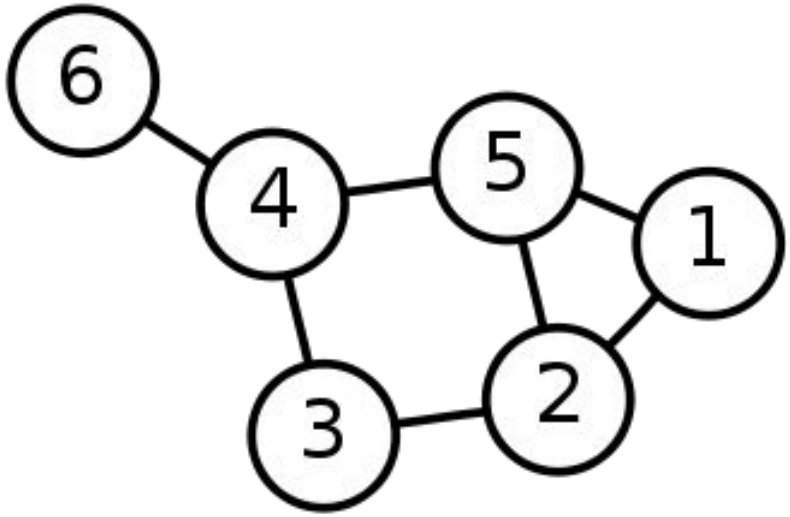


Single Source Shortest Path: Dijkstra's Algorithm

https://www.youtube.com/watch?v=YJfApSMbwaE

Single-Source Shortest Path Problem

Single-Source Shortest Path Problem - The problem of finding shortest paths from a source vertex *v* to all other vertices in the graph.



Dijkstra's algorithm

Dijkstra's algorithm - is a solution to the single-source shortest path problem in graph theory.

Works on both directed and undirected graphs. However, all edges must have nonnegative weights.

Approach: Greedy

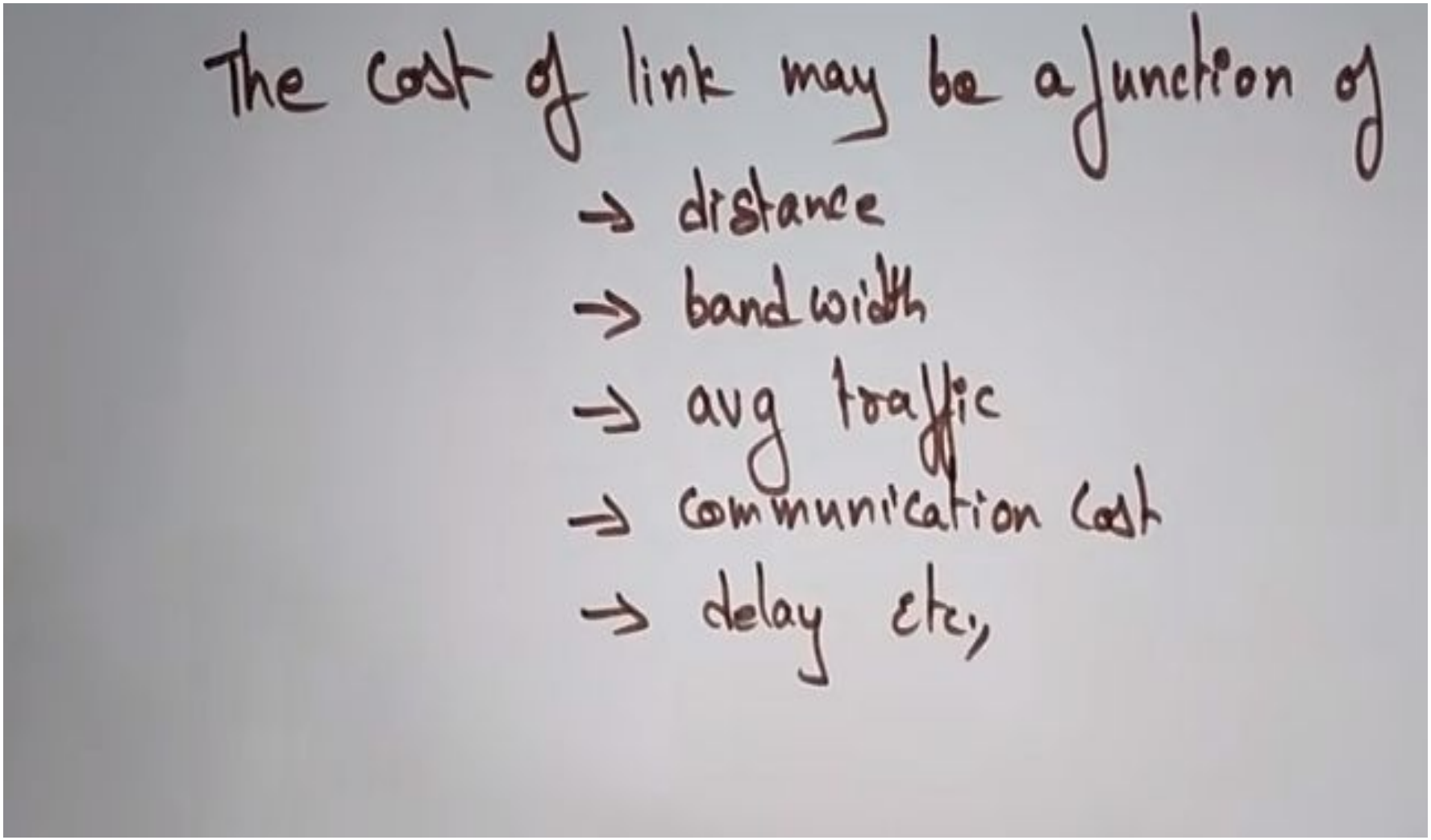
Input: Weighted graph G={E,V} and source vertex $v \in V$, such that all edge weights are nonnegative

Output: Lengths of shortest paths (or the shortest paths themselves) from a given source vertex $v \in V$ to all other vertices

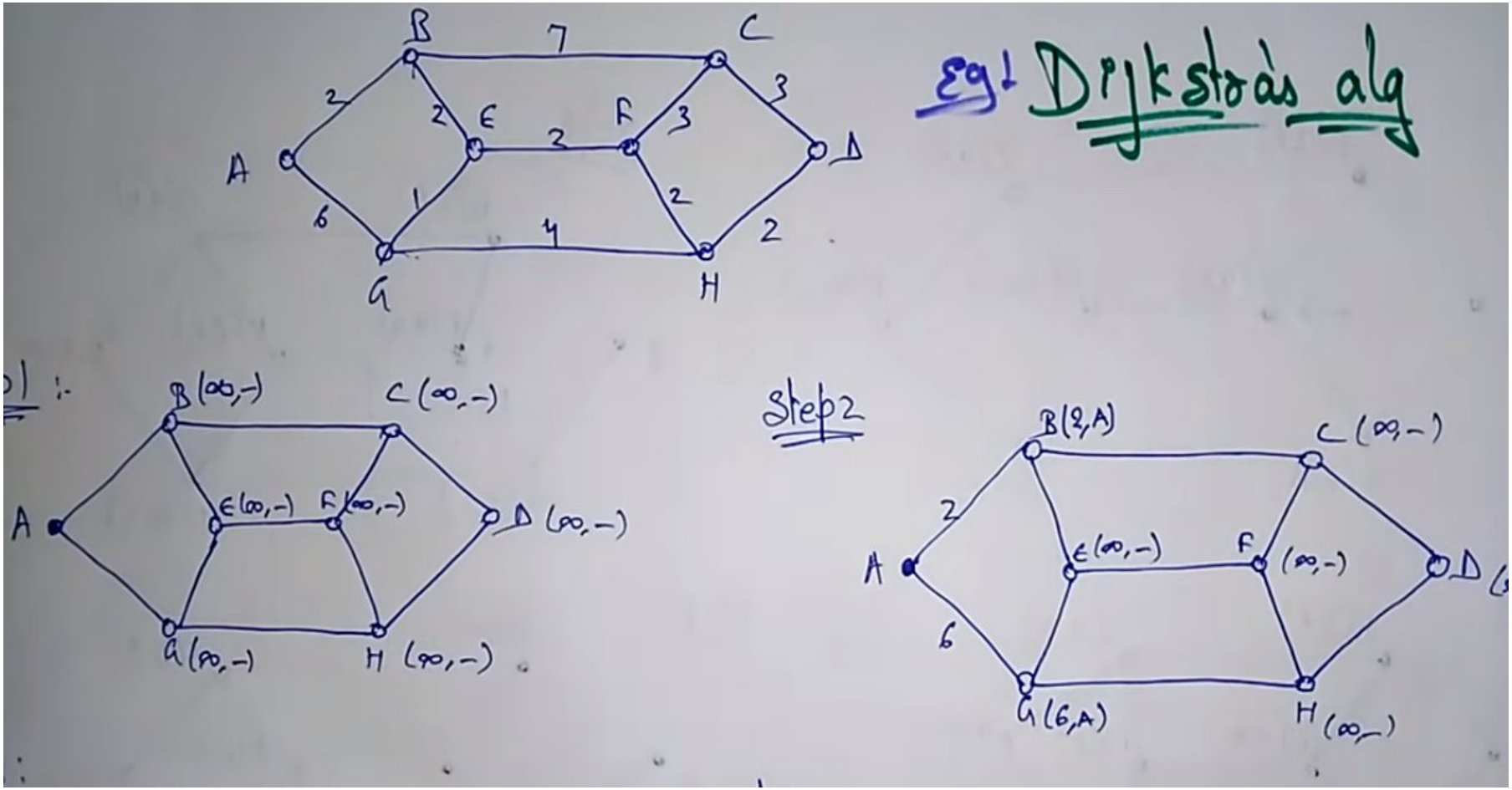
Dijkstra's algorithm - Pseudocode

```
dist[s] ←0                          (distance to source vertex is zero)
for  all v ∈ V–{s}
       do  dist[v] ←∞               (set all other distances to infinity)
S←∅                                 (S, the set of visited vertices is initially empty)
Q←V                                 (Q, the queue initially contains all vertices)
while Q ≠∅                          (while the queue is not empty)
do   u ← mindistance(Q,dist)     (select the element of Q with the min. distance)
     S←S∪{u}                        (add u to list of visited vertices)
      for all v ∈ neighbors[u]
            do  if   dist[v] > dist[u] + w(u, v)        (if new shortest path found)
                     then      d[v] ←d[u] + w(u, v)   (set new value of shortest path)
            (if desired, add traceback code)
return dist
```

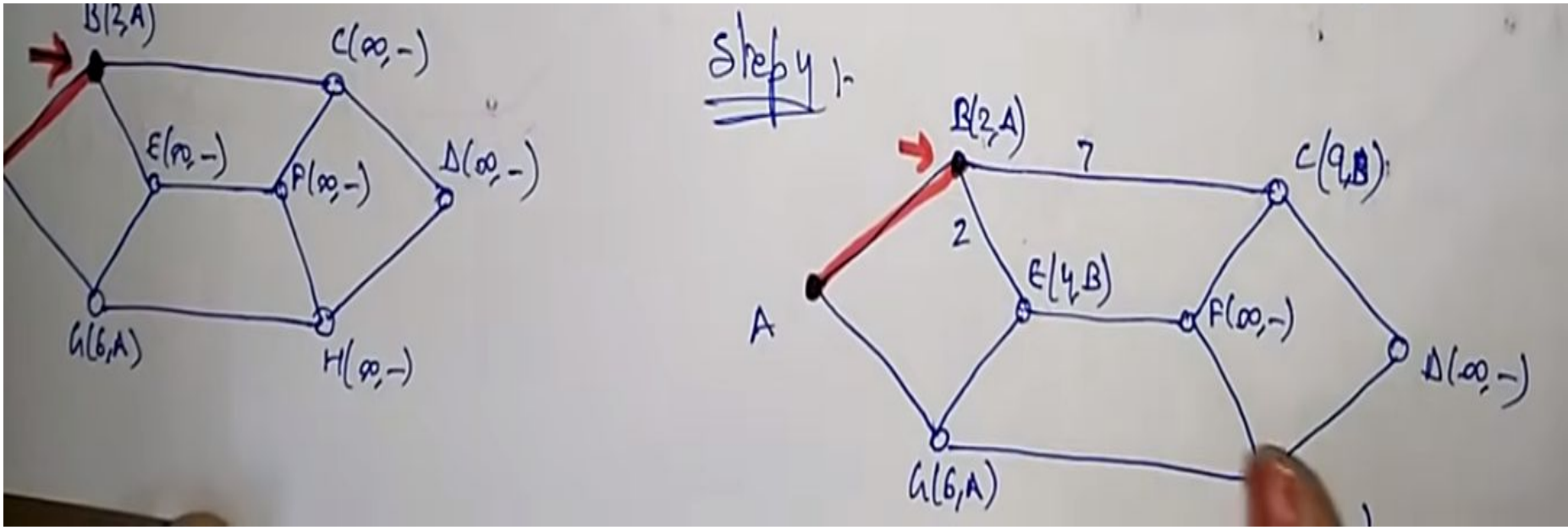
Example


The cost of link may be a function of
→ distance
→ bandwidth
→ avg traffic
→ communication cost
→ delay etc,

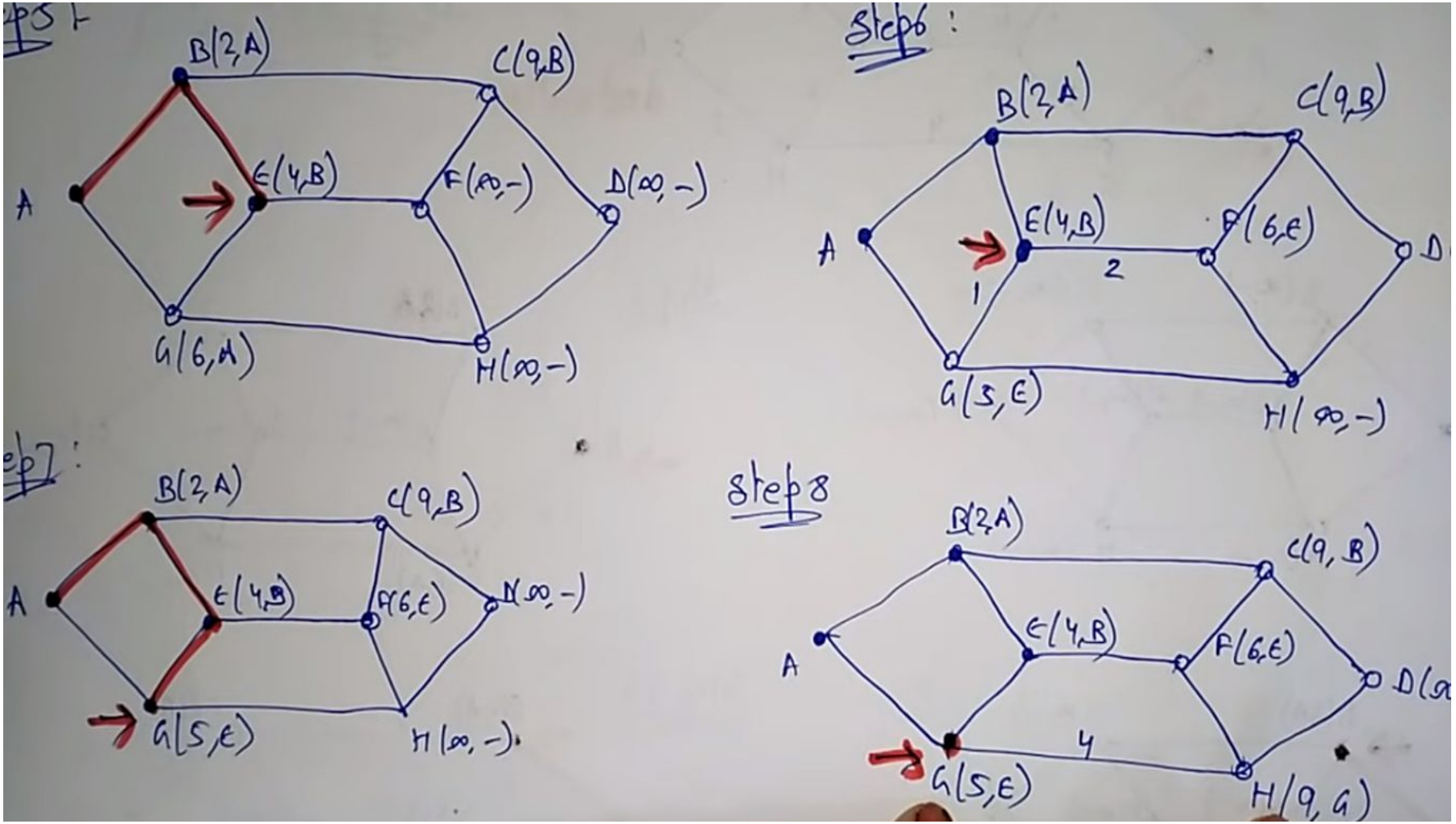
Example


Eg! Dijkstra's alg
B
C
A
E
F
D
G
H
7
2
2
3
2
3
1
6
4
2
Step2
B(∞,-)
C(∞,-)
A
E(∞,-)
F(∞,-)
D(∞,-)
G(∞,-)
H(∞,-)
B(2,A)
C(∞,-)
E(∞,-)
F
(∞,-)
G(6,A)
H(∞,-)

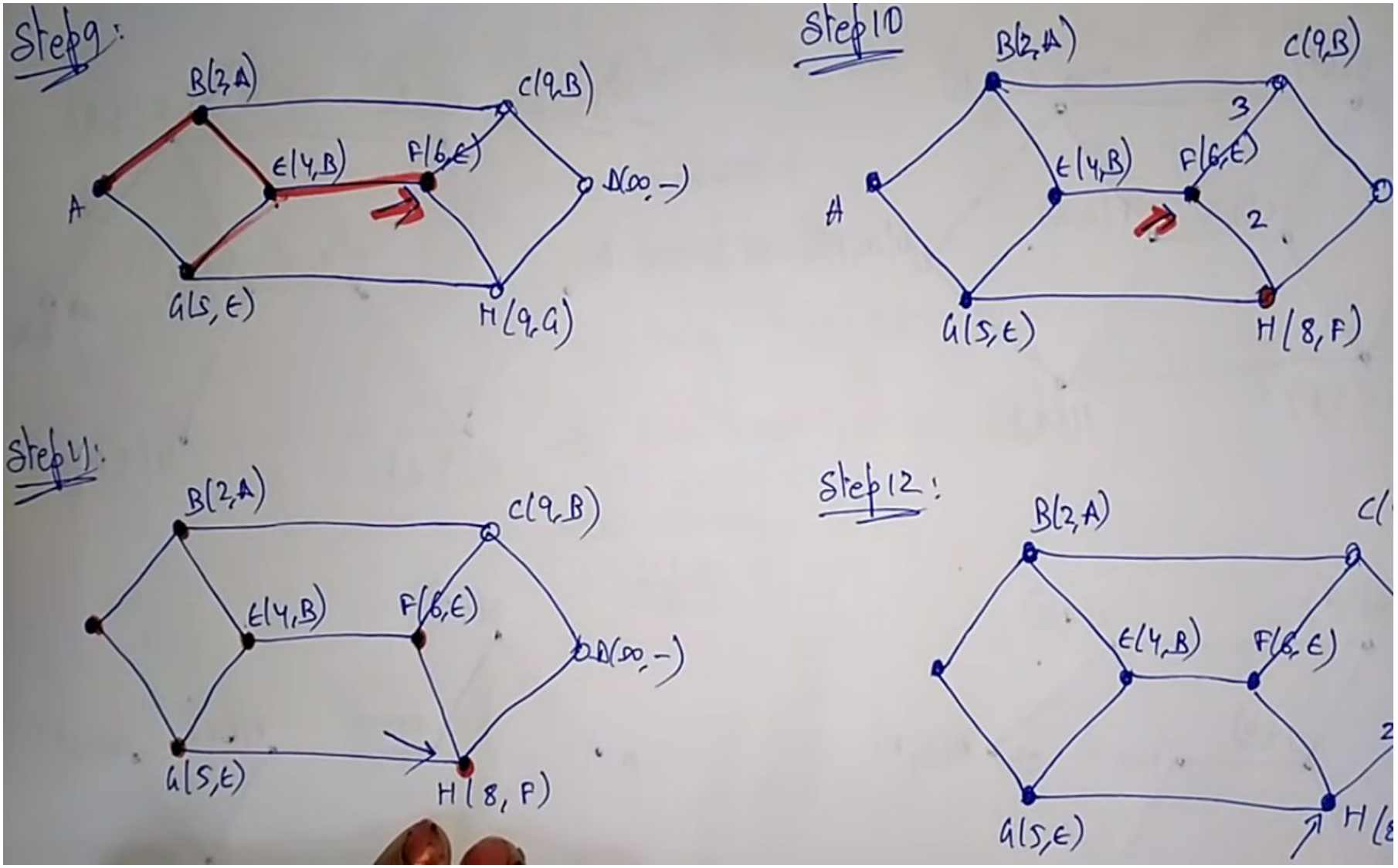
Example



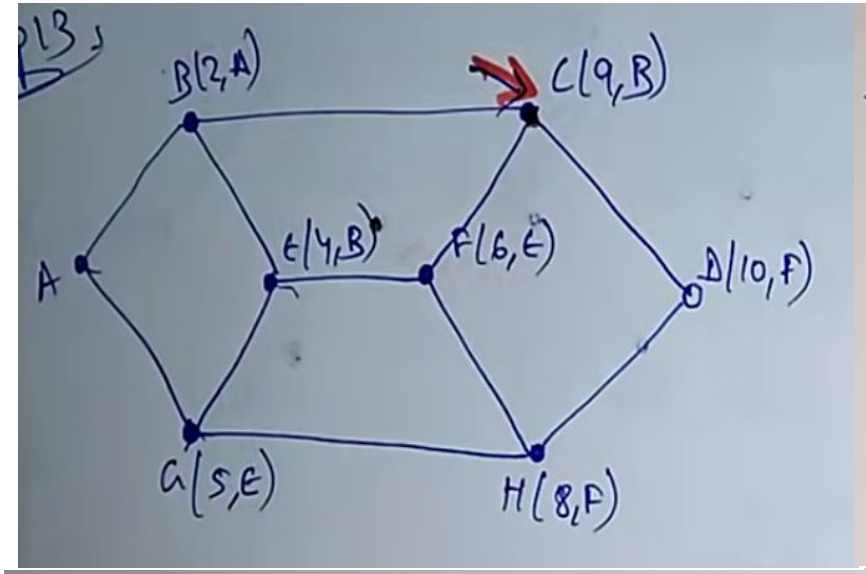
Example

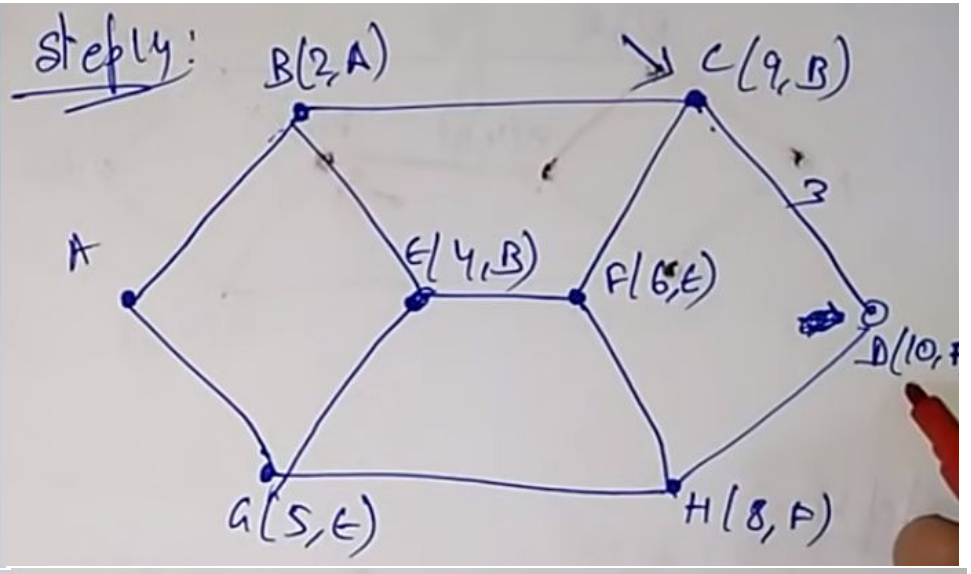

B(2,A)
C(9,B)
A
E(4,B)
F(∞,-)
D(∞,-)
G(6,H)
H(∞,-)
Step6 :
B(2,A)
C(9,B)
A
E(4,B)
F(6,E)
1
2
G(5,E)
H(∞,-)
B(2,A)
C(9,B)
A
E(4,B)
F(6,E)
G(5,E)
H(∞,-)
step8
B(2,A)
C(9,B)
A
E(4,B)
F(6,E)
4
G(5,E)
H(9,G)

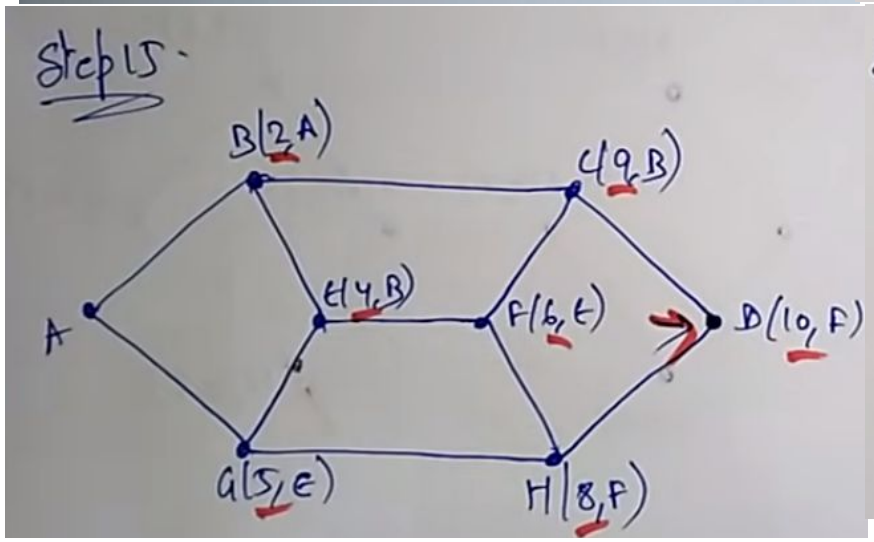
Example

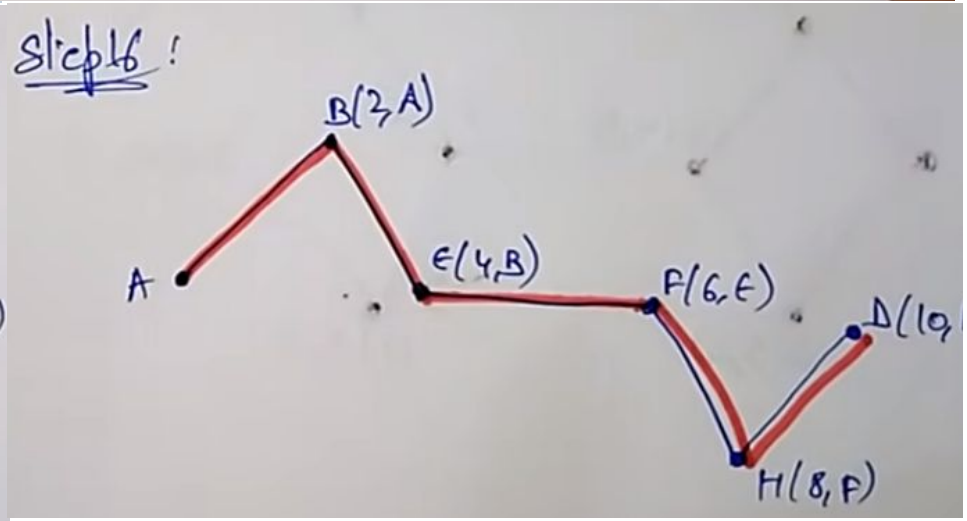

Step 9:
B(2,A)
C(9,B)
E(4,B)
F(6,E)
A
D(∞,–)
G(5,E)
H(9,G)
Step 10
B(2,A)
C(9,B)
3
E(4,B)
F(6,E)
A
2
G(5,E)
H(8,F)
Step 11:
B(2,A)
C(9,B)
E(4,B)
F(6,E)
D(∞,–)
G(5,E)
H(8,F)
Step 12:
B(2,A)
E(4,B)
F(6,E)
G(5,E)

Example


B(2,A)
C(9,B)
A
E(4,B)
F(6,E)
D(10,F)
G(5,E)
H(8,F)

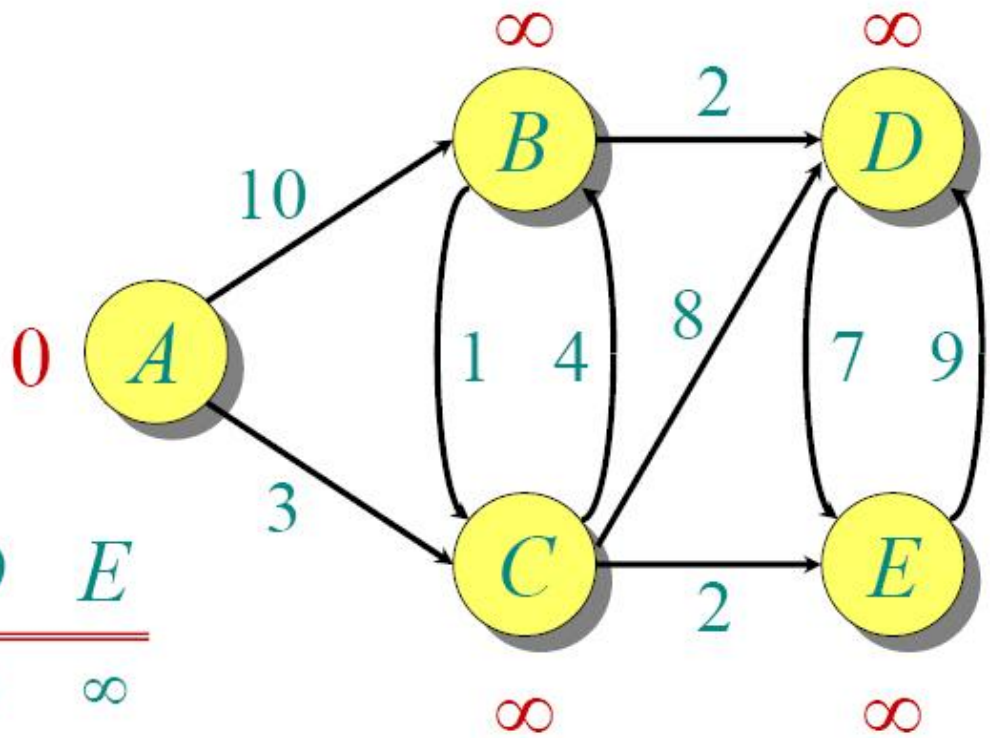

Step 14:
B(2,A)
C(9,B)
A
E(4,B)
F(6,E)
G(5,E)
H(8,F)


Step 15:
B(2,A)
C(9,B)
A
E(4,B)
F(6,E)
D(10,F)
G(5,E)
H(8,F)


Step 16:
B(2,A)
A
E(4,B)
F(6,E)
H(8,F)

Dijkstra Animated Example

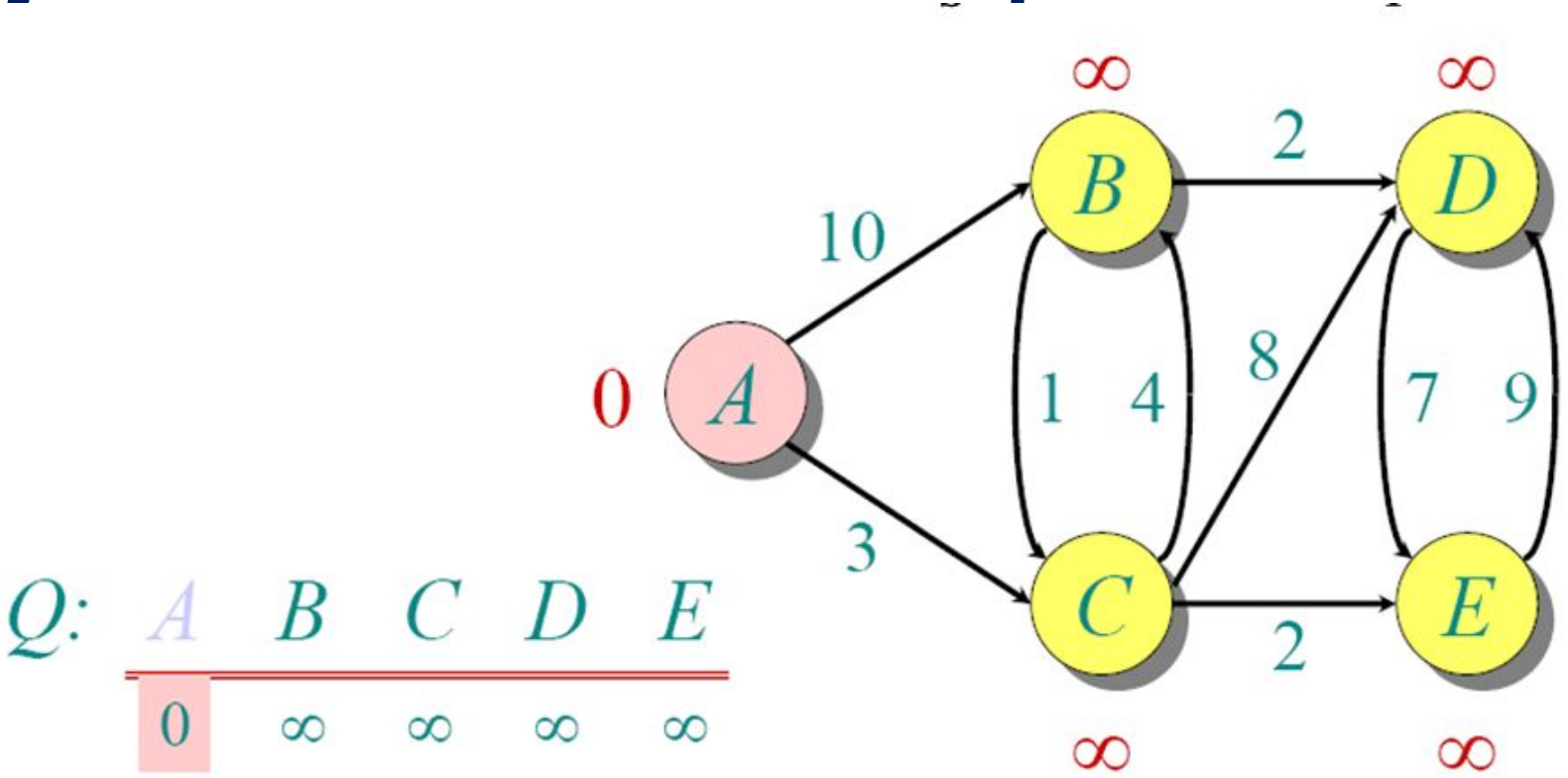
Initialize:



Q:	A	B	C	D	E
	0	∞	∞	∞	∞

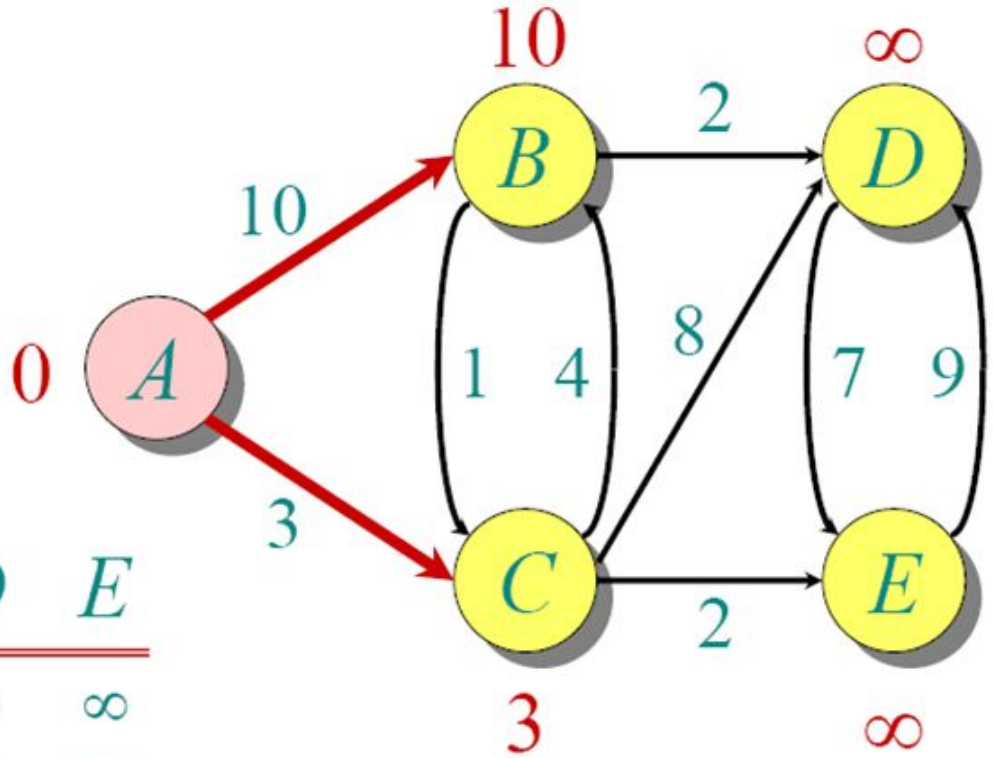
S: {}

Dijkstra Animated Example



Q:	A	B	C	D	E
	0	∞	∞	∞	∞

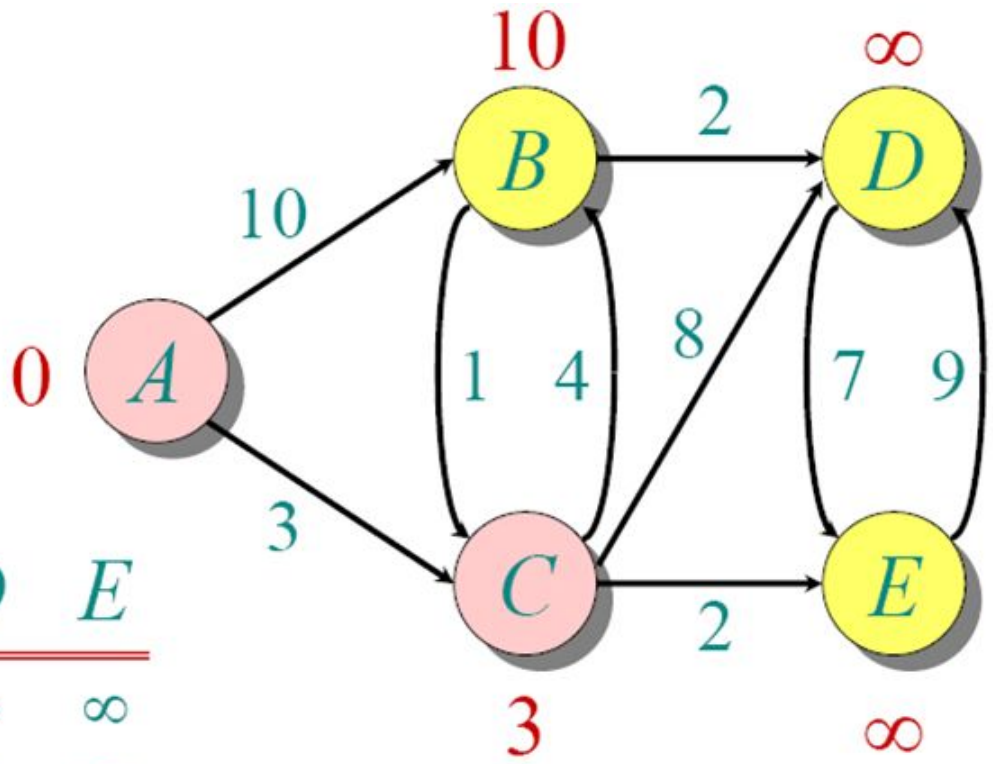
Dijkstra Animated Example



Q:	A	B	C	D	E
	0	∞	∞	∞	∞
		10	3	∞	∞

$S: \{ A \}$

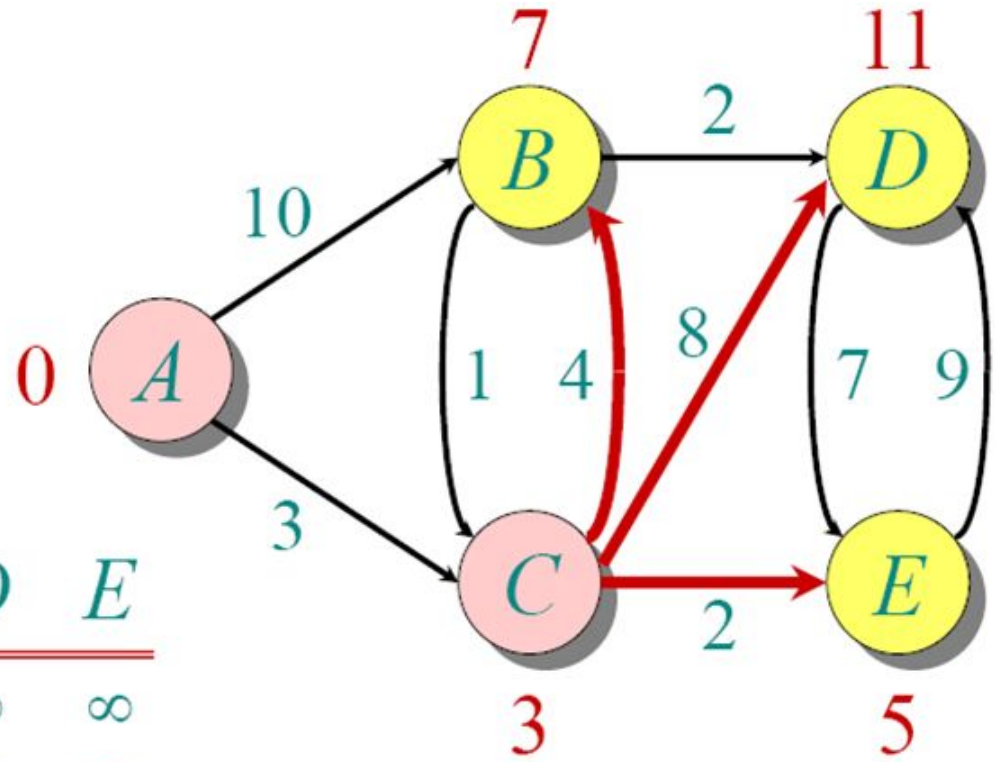
Dijkstra Animated Example



Q:	A	B	C	D	E
	0	∞	∞	∞	∞
		10	3	∞	∞

$S: \{ A, C \}$

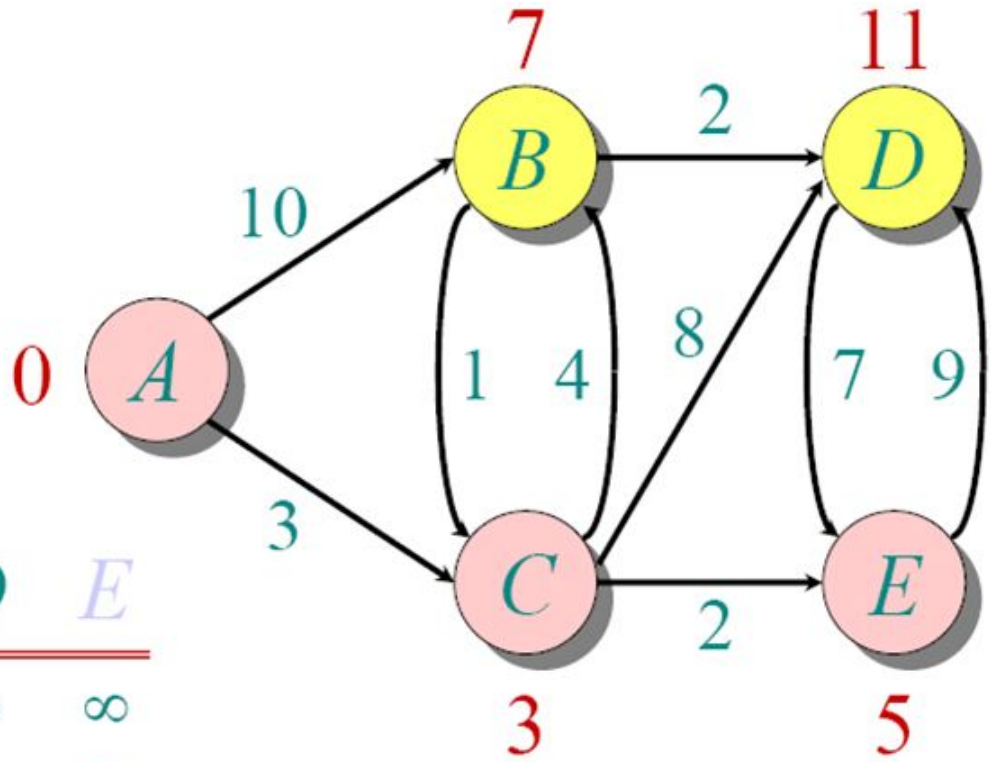
Dijkstra Animated Example



Q:	A	B	C	D	E
	0	∞	∞	∞	∞
		10	3	∞	∞
		7		11	5

$S: \{ A, C \}$

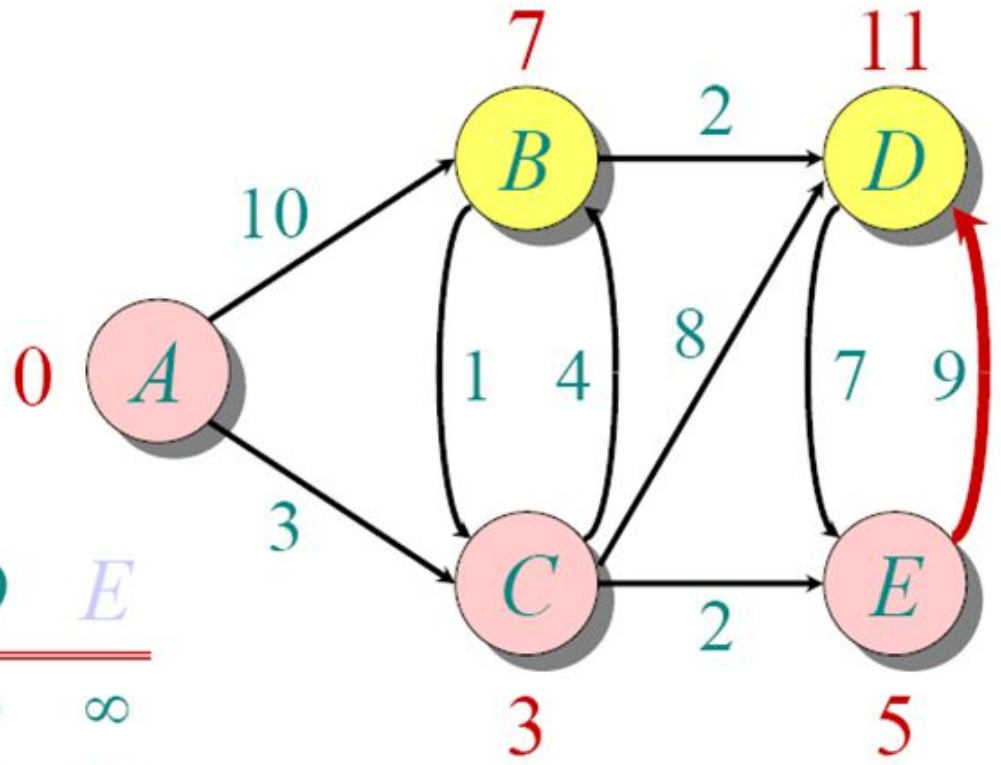
Dijkstra Animated Example



Q:	A	B	C	D	E
	0	∞	∞	∞	∞
		10	3	∞	∞
		7		11	5

$S: \{ A, C, E \}$

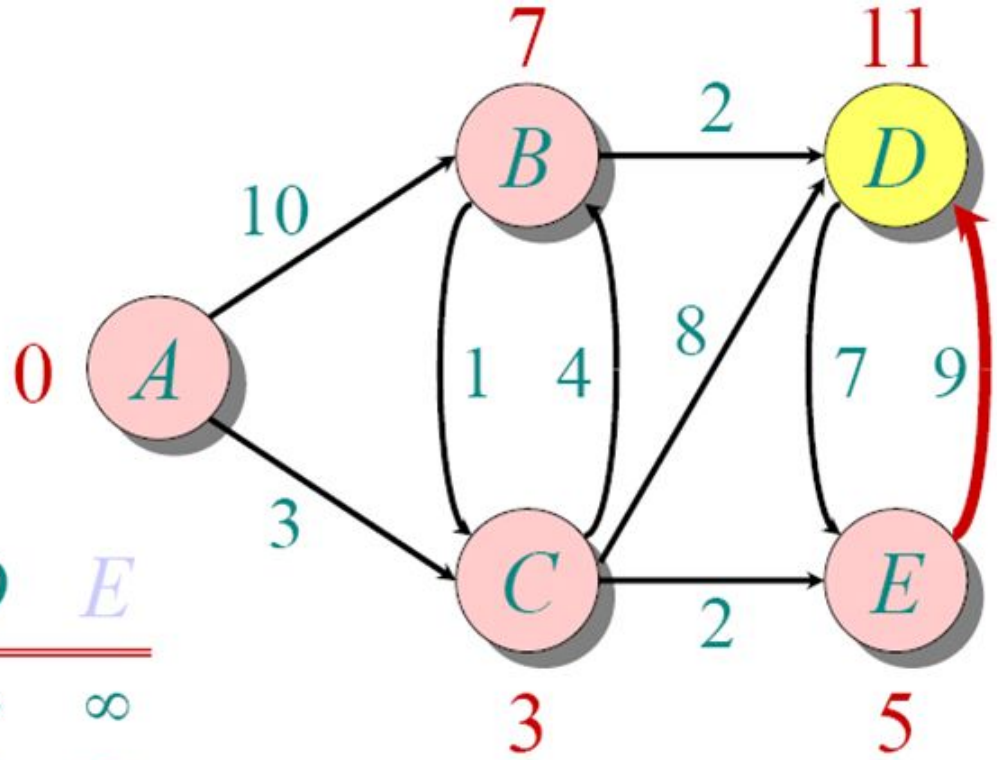
Dijkstra Animated Example



Q:	A	B	C	D	E
	0	∞	∞	∞	∞
		10	3	∞	∞
		7		11	5
		7		11	

$S: \{ A, C, E \}$

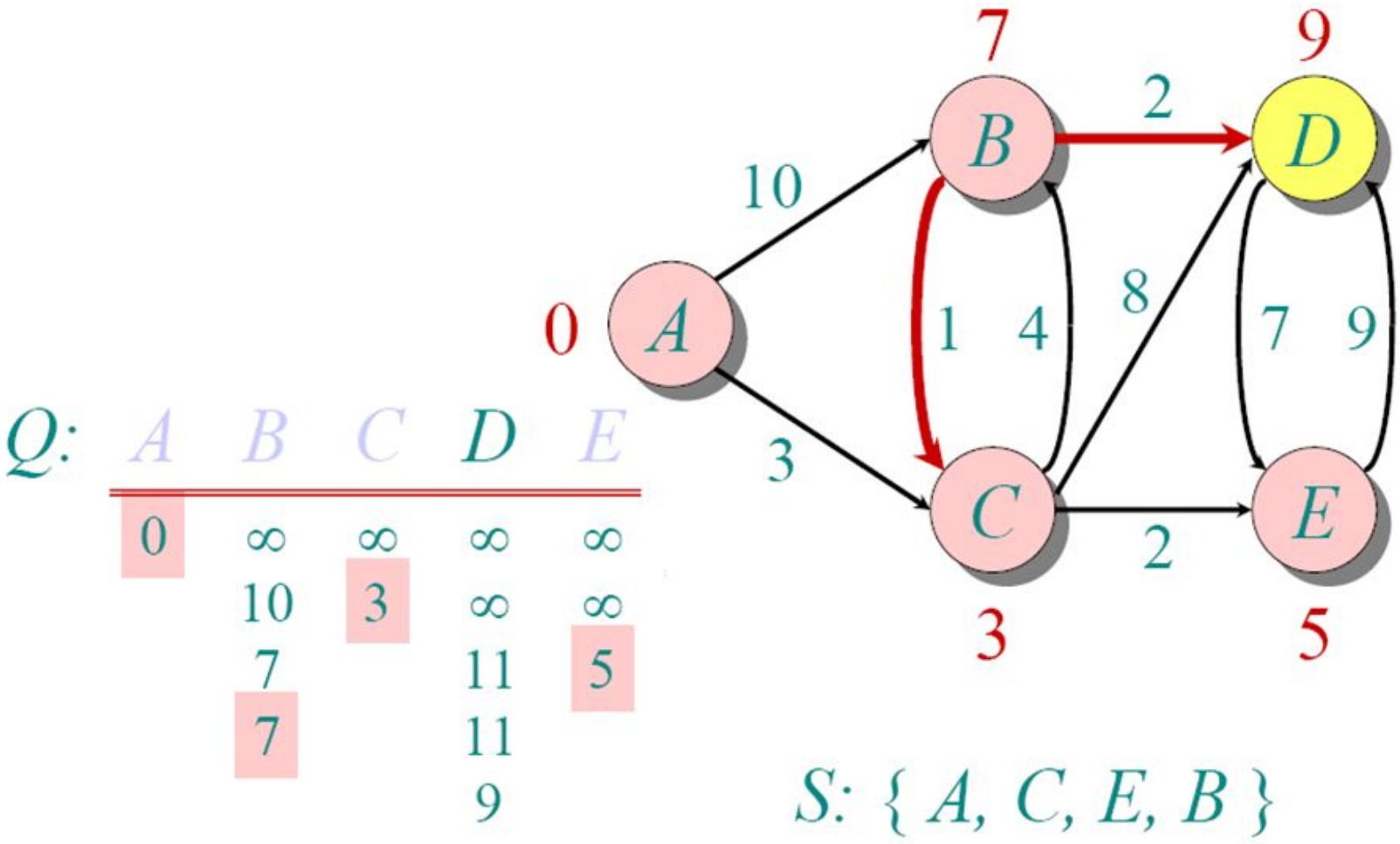
Dijkstra Animated Example



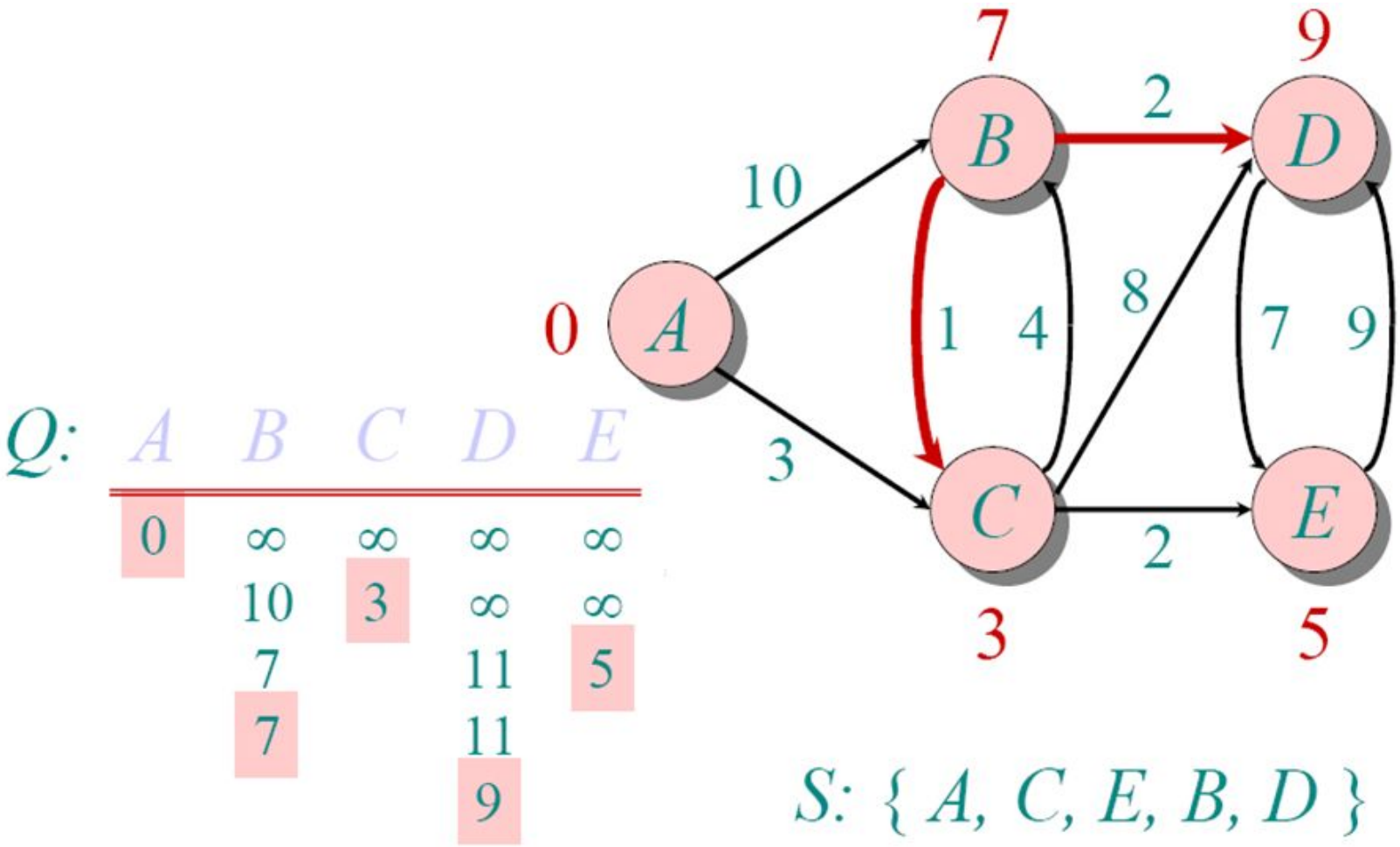
Q:	A	B	C	D	E
	0	∞	∞	∞	∞
		10	3	∞	∞
		7		11	5
		7		11	

S: { A, C, E, B }

Dijkstra Animated Example



Dijkstra Animated Example



Implementations and Running Times

The simplest implementation is to store vertices in an array or linked list. This will produce a running time of

$O(|V|^2 + |E|)$

For sparse graphs, or graphs with very few edges and many nodes, it can be implemented more efficiently storing the graph in an adjacency list using a binary heap or priority queue. This will produce a running time of

$O((|E|+|V|) \log |V|)$

Dijkstra's Algorithm - Why It Works

- As with all greedy algorithms, we need to make sure that it is a correct algorithm (e.g., it *always* returns the right solution if it is given correct input).

- A formal proof would take longer than this presentation, but we can understand how the argument works intuitively.

- If you can't sleep unless you see a proof, see the second reference or ask us where you can find it.

Dijkstra's Algorithm - Why It Works

- To understand how it works, we'll go over the previous example again. However, we need two mathematical results first:

- **Lemma 1**: Triangle inequality
  If δ(u,v) is the shortest path length between u and v,
  $\delta(u,v) \leq \delta(u,x) + \delta(x,v)$
- **Lemma 2**:
  The subpath of any shortest path is itself a shortest path.

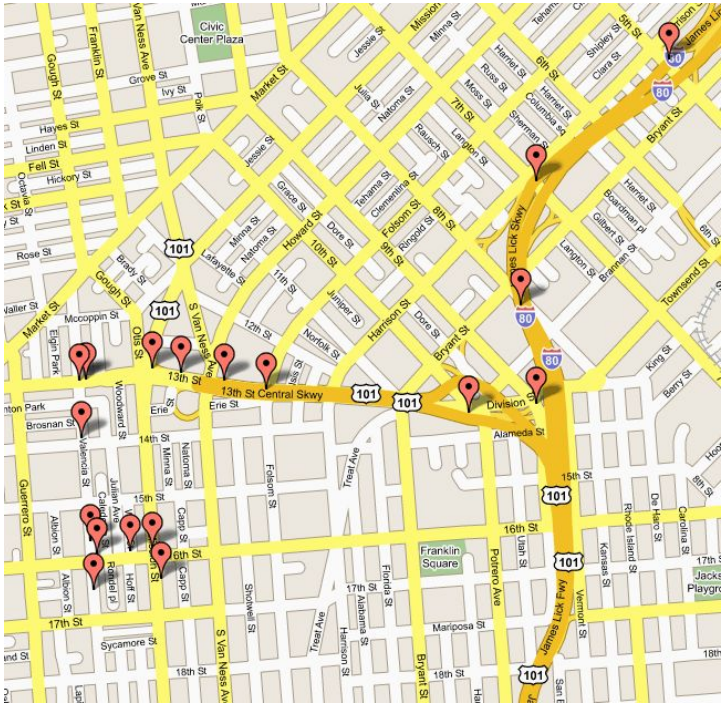
- The key is to understand why we can claim that anytime we put a new vertex in S, we can say that we already know the shortest path to it.
- Now, back to the example…

Dijkstra's Algorithm - Why use it?

- As mentioned, Dijkstra's algorithm calculates the shortest path to every vertex.
- However, it is about as computationally expensive to calculate the shortest path from vertex *u* to every vertex using Dijkstra's as it is to calculate the shortest path to some particular vertex *v*.
- Therefore, anytime we want to know the optimal path to some other vertex from a determined origin, we can use Dijkstra's algorithm.

Applications of Dijkstra's Algorithm

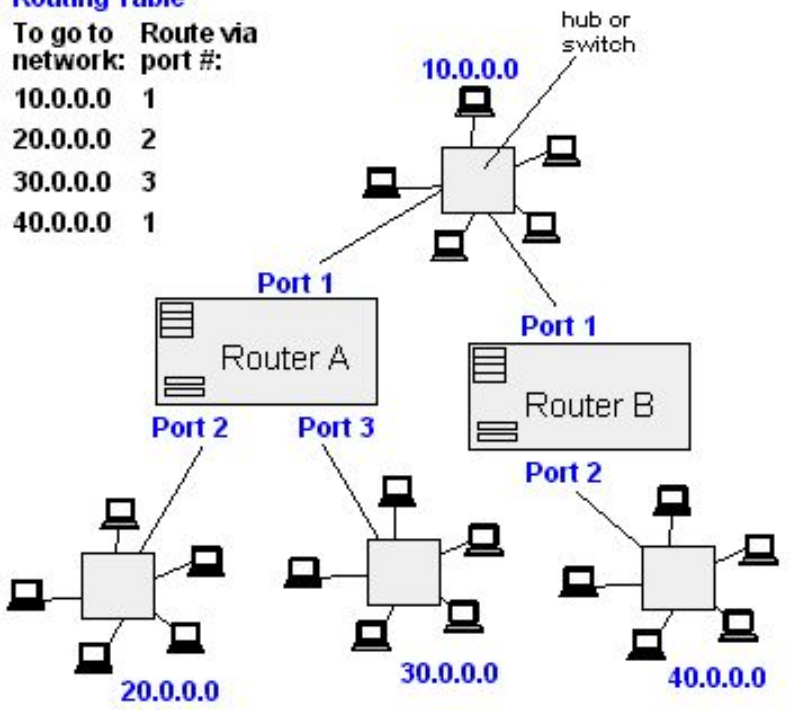
- Traffic Information Systems are most prominent use
- Mapping (Map Quest, Google Maps)
- Routing Systems



From Computer Desktop Encyclopedia
© 1998 The Computer Language Co. Inc.

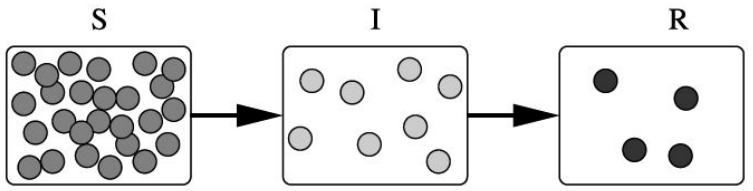
Router A
Routing Table

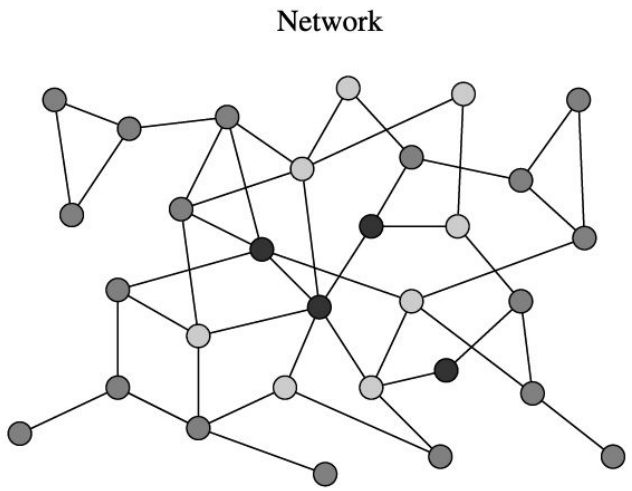
To go to network:	Route via port #:
10.0.0.0	1
20.0.0.0	2
30.0.0.0	3
40.0.0.0	1



Applications of Dijkstra's Algorithm

- One particularly relevant this week: epidemiology
- Prof. Lauren Meyers (Biology Dept.) uses networks to model the spread of infectious diseases and design prevention and response strategies.
- Vertices represent individuals, and edges their possible contacts. It is useful to calculate how a particular individual is connected to others.
- Knowing the shortest path lengths to other individuals can be a relevant indicator of the potential of a particular individual to infect others.





References

- Dijkstra's original paper: E. W. Dijkstra. (1959) *A Note on Two Problems in Connection with Graphs.* Numerische Mathematik, 1. 269-271.
- MIT OpenCourseware, 6.046J Introduction to Algorithms. < http://ocw.mit.edu/OcwWeb/Electrical-Engineering-and-Computer-Science/6-046JFall-2005/CourseHome/> Accessed 4/25/09
- Meyers, L.A. (2007) Contact network epidemiology: Bond percolation applied to infectious disease prediction and control. *Bulletin of the American Mathematical Society* **44**: 63-86.
- Department of Mathematics, University of Melbourne. *Dijkstra's Algorithm.* <http://www.ms.unimelb.edu.au/~moshe/620-261/dijkstra/dijkstra.html > Accessed 4/25/09

END